UNIT-IV

DIGITAL INTEGRATED CIRCUITS

Classification of Integrated Circuits

1.1 CLASSIFICATION OF IC'S:

Integrated Circuitis a miniature, low cost electronic circuit consisting of active and passive components that are irreparably joined together on a single crystal chip of silicon.

CLASSIFICATION:

1.1.1 Based on mode of operation

a. Digital IC's
b. Linear IC's

Digital IC's: Digital IC's are complete functioning logic networks that are equivalents of basic transistor logic circuits.
Ex:- gates ,counters, multiplexers, demultiplexers, shift registers.

Linear IC's: Linear IC's are equivalents of discrete transistor networks, such as amplifiers, filters, frequency multipliers, and modulators that often require additional external components for satisfactory operation

1.1.2. Based on fabrication

a. Monolithic IC's
b. Hybrid IC's

a. **Monolithic IC's** : In monolithic ICs all components (active and passive) are forme simultaneously by a diffusion process. Then a metallization process is used i interconnecting these components to form the desired circuit.
b. **Hybrid IC's:** .In hybrid ICs, passive components (such as resistors and capacitors) and tl interconnections between them are formed on an insulating substrate. The substrate is use as a chassis for the integrated components. Active components such as transistors and diode as wellas monolithic integrated circuits, are then connected to form a complete circuit.

1.1.3. Based on number of components integrated on IC's

a. SSI <10 components
b. MSI <100 components
c. LSI >100 components
d. VLSI >1000 components

1.2. CHIP SIZE AND CIRCUIT COMPLEXITY:

Invention of transistor	1948
Development of silicon transistor	1955-1959
Silicon planar technology	1959
First IC,Small Scale Integration(SSI),3 to 30 gates/chip	1960
Medium Scale Integration (MSI),30 to 300 gates/chip	1965-1970
Large Scale Integration(LSI),300 to 3000 gates/chip	1970-1975
Very Large Scale Integration(VLSI) more than 3000 gates/chip	1975

An integrated circuit or monolithic integrated circuit (also referred to as IC, chip, or microchip) is an electronic circuit. Integrated circuits are used in virtually all electronic equipment today and have revolutionized the world of electronics. Computers, cell phones, and other digital appliances are now inextricable parts of the structure of modern societies, made possible by the low cost of production of integrated circuits. There are two main advantages of ICs over discrete circuits: cost and performance. Cost is low because the chips, with all their components, are printed as a unit by photolithography rather than being constructed one transistor at a time. Furthermore, much less material is used to construct a packaged IC die than to construct a discrete circuit.

Performance is high because the components switch quickly and consume little power (compared to their discrete counterparts) as a result of the small size and close proximity of the components.

6.2 CLASSIFICATION OF INTEGRATED CIRCUITS

Classification of ICs based on complexity:

- Small Scale Integration or (SSI) - Contain up to 10 transistors or a few gates within a single package such as AND, OR, NOT gates.
- Medium Scale Integration or (MSI) - between 10 and 100 transistors or tens of gates within a single package and perform digital operations such as adders, decoders, counters, flip-flops and multiplexers.
- Large Scale Integration or (LSI) - between 100 and 1,000 transistors or hundreds of gates and perform specific digital operations such as I/O chips, memory, arithmetic and logic units.
- Very-Large Scale Integration or (VLSI) - between 1,000 and 10,000 transistors or thousands of gates and perform computational operations such as processors, large memory arrays and programmable logic devices.
- Super-Large Scale Integration or (SLSI) - between 10,000 and 100,000 transistors within a single package and perform computational operations such as microprocessor chips, micro-controllers, basic PICs and calculators.
- Ultra-Large Scale Integration or (ULSI) - more than 1 million transistors - the big boys that are used in computers CPUs, GPUs, video processors, micro-controllers, FPGAs and complex PICs.

While the "ultra large scale" ULSI classification is less well used, another level of integration which represents the complexity of the Integrated Circuit is known as the **System-on-Chip** or (**SOC**) for short. Here the individual components such as the microprocessor, memory, peripherals, I/O logic etc, are all produced on a single piece of silicon and which represents a whole electronic system within one single chip, literally putting the word "integrated" into integrated circuit. These chips are generally used in mobile phones, digital cameras, micro-controllers, PICs and robotic applications, and which can contain up to 100 million individual silicon-CMOS transistor gates within one single package.

Classification of integrated circuits based on the signal processed:

- DIGITAL INTEGRATED CIRCUITS

Digital integrated circuits, primarily used to build computer systems, also occur in cellular phones, stereos and televisions. Digital integrated circuits include microprocessors, microcontrollers and logic circuits. They perform mathematical calculations, direct the flow of data and make decisions based on Boolean logic principles.The Boolean system used centers on on two numbers: 0 and 1.

- ANALOG INTEGRATED CIRCUITS

Analog integrated circuits most commonly make up a part of power supplies, instruments and communications. In these applications, analog integrated circuits amplify, filter and modify electrical signals. In cellular phones, they amplify and filter the incoming signal from the phone's antenna. The sound encoded into that signal has a low amplitude level; after the circuit filters the sound signal from the incoming signal, the circuit amplifies the sound signal and sends it to the speaker in your cell phone, allowing you to hear the voice on the other end.

- MIXED-SIGNAL INTEGRATED CIRCUITS

Mixed-signal circuits occur in cellular phones, instrumentation, motor and industrial control applications. These circuits convert digital signals to analog signals, which in turn set the speed of motors, the brightness of lights and the temperature of heaters, for example. They also convert digital signals to sound waveforms, allowing for the design of digital musical instruments such as electronic organs and computer keyboards capable of playing music.

Mixed-signal integrated circuits also convert analog signals to digital signals. They will convert analog voltage levels to digital number representations of the voltage level of the signals. Digital integrated circuits then perform mathematical calculations on these numbers.

- MEMORY-INTEGRATED CIRCUITS

Though primarily used in computer systems, memory-integrated circuits also occur in cellular phones, stereos and televisions. A computer system may include 20 to 40 memory chips, while other types of electronic systems may contain just a few.

Memory circuits store information, or data, as two numbers: 0 and 1. Digital integrated circuits will often retrieve these numbers from memory and perform calculations with them, then save the calculation result a memory chip's data storage locations. The more data it accesses---pictures, sound and text---the more memory an electronic system will require.

Combinational Logic ICs – Specifications and Applications of TTL-74XX & CMOS 40XX Series ICs

7400 Series TTL: The 7400 series TTL logic integrated circuits provided the basis for most of the logic circuits of the time and they are still available today.

The 7400 series of TTL logic integrated circuits was introduced in October 1966. The 7400 series logic TTL chips spawned a series of other derivative logic families offering slightly different characteristics: high speed, low power, etc. However the standard parameters remained the same: logic function (a 7416 and a 74LS16 had the same function; they were pin compatible, etc.

The 7400 series TTL chips remained in use for many years. They have long been superseded by other 74xx00 logic families, but they have been so successful that the basic concept has remained the same.

7400 series main features

Some of the main or highlight features and specifications for the 7400 series logic family are detailed below:

Summary of 7400 Series TTL Key Parameters

Parameter	Specification
Supply voltage	Nominal 5V (4.75 - 5.25)
Max toggle speed	25 MHz
Propagation delay per gate	Typically 10 ns
Power consumption per gate	10 mW

7400 series output stages

There are three types of output stage that 7400 series logic may possess.

- ***Totem pole:*** This output is the standard output format for 7400 series logic chips. It comprises two transistors and enables very fast switching times to be achieved.

The advantages of using the totem-pole output are threefold:

- Low power consumption
- Fast switching
- Low output impedance

The **4000 series** is a range of **CMOS ICs** introduced as a lower-power and more versatile alternative to the 7400 **series** in common use. There are many **ICs** in the **4000 series**, from simple logic gates to counters and many more.

General characteristics of 4000 series CMOS ICs

- **Supply:** 3 to 15V, small fluctuations are tolerated.
- **Inputs** have very high impedance (resistance), this is good because it means they will not affect the part of the circuit where they are connected. However, it also means that unconnected inputs can easily pick up electrical noise and rapidly change between high and low states in an unpredictable way. This is likely to make the IC behave erratically and it will significantly increase the supply current. To prevent problems **all unused inputs MUST be connected to the supply (either +Vs or 0V)**, this applies even if that part of the IC is not being used in the circuit!
- **Outputs** can sink and source only about 1mA if you wish to maintain the correct output voltage to drive CMOS inputs. If there is no need to drive any inputs the maximum current is about 5mA with a 6V supply, or 10mA with a 9V supply (just enough to light an LED). To switch larger currents you can connect a transistor.
- **Fan-out:** one output can drive up to 50 inputs.
- **Gate propagation time**: typically 30ns for a signal to travel through a gate with a 9V supply, it takes a longer time at lower supply voltages.
- **Frequency**: up to 1MHz, above that the 74 series is a better choice.
- **Power consumption** (of the IC itself) is very low, a few µW. It is much greater at high frequencies, a few mW at 1MHz for example.

Quad 2-input gates

- 4001 quad 2-input NOR
- 4011 quad 2-input NAND
- 4030 quad 2-input EX-OR (now obsolete)
- 4070 quad 2-input EX-OR
- 4071 quad 2-input OR
- 4077 quad 2-input EX-NOR
- 4081 quad 2-input AND
- 4093 quad 2-input NAND with Schmitt trigger inputs

The 4093 has Schmitt trigger inputs to provide good noise immunity. They are ideal for slowly changing or noisy signals. The hysteresis is about 0.5V with a 4.5V supply and almost 2V with a 9V supply.

Code Converters:

The Gray Code is unweighted and is not an arithmetic code: that is, there are no specific weights asssigned to the bit positions. The important feature of the Gray Code is that it exhibits only a single bit change from one code word to the next in sequence. This property is important in many applications, such as shaft position encoders, where error susceptibility increases with the number of bit changes between adjacent numbers in a sequence.

To convert a binary number to a Gray Code number, the following rules apply.

1. The most significant digit (Left Most Bit) in the Gray Code is the same as the corresponding digit in the binary number.

2. Going from left to right, add each adjacent pair of binary digits to get the next Gray code digit, regradless carries.

For instance - Let us convert the binary number 1010 to Gray Code.

Step 1 - The left most Gray digit is the same as the left most binary

```
1    0    1    0    Binary
↓
1                   Gray
```

Step 2 - Add the left most binary digit to the adjacent one.

```
1 + 0    1    0    Binary
    ↓
1   1              Gray
```

Step 3 - Add the next adjacent pair

```
1    0  + 1    0    Binary
        ↓
1    1    1         Gray
```

Step 4 -Add the last adjacent pair

```
1    0    1  + 0    Binary
             ↓
1    1    1    1    Gray
```

The conversion is now complete and the Gray Code is 1111.

Steps to design the converter

1. Design a converter by the following procedures:

a. Write down the truth table of both input and output bits of the converter.

b. Apply Karnaugh Map to look for the minimized logic expression for the output bits.

c. Implement the logic gates by using Circuit Maker.

Example:

For Binary to Gray Code Converter, binary bits are input and gray code bits are output. So first write the truth table for binary bits and gray code. Then k-map for the all bits of gray code, find the simplified expression for each bit of gray code. Then design the logical circuit.

Truth Table

Decimal	Binary				Gray			
	A	B	C	D	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	0	0	0	1	1
3	0	0	1	1	0	0	1	0
4	0	1	0	0	0	1	1	0
5	0	1	0	1	0	1	1	1
6	0	1	1	0	0	1	0	1
7	0	1	1	1	0	1	0	0
8	1	0	0	0	1	1	0	0
9	1	0	0	1	1	1	0	1
10	1	0	1	0	1	1	1	1
11	1	0	1	1	1	1	1	0
12	1	1	0	0	1	0	1	0
13	1	1	0	1	1	0	1	1
14	1	1	1	0	1	0	0	1
15	1	1	1	1	1	0	0	0

K-Map for each bit of Gray code

For practical consideration code conversions are made for each bit.

(a) For Y_3, the Karnaugh map can draw as follow.

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	0	0	0	0
AB	1	1	1	1
$A\bar{B}$	1	1	1	1

(a)

By minimization, $Y_3 = A$

(b) For Y_2, the Karnaugh map can draw as f

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	0	0
$\bar{A}B$	1	1	1	1
AB	0	0	0	0
$A\bar{B}$	1	1	1	1

(b)

By minimization, $Y_2 = A \oplus B$

(c) For Y_1, the Karnaugh map can draw as follow.

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	0	1	1
$\bar{A}B$	1	1	0	0
AB	1	1	0	0
$A\bar{B}$	0	0	1	1

(a)

By minimization, $Y_1 = B \oplus C$

(d) For Y_0, the Karnaugh map can draw as follow.

	$\bar{C}\bar{D}$	$\bar{C}D$	CD	$C\bar{D}$
$\bar{A}\bar{B}$	0	1	0	1
$\bar{A}B$	0	1	0	1
AB	0	1	0	1
$A\bar{B}$	0	1	0	1

(b)

By minimization, $Y_0 = C \oplus D$

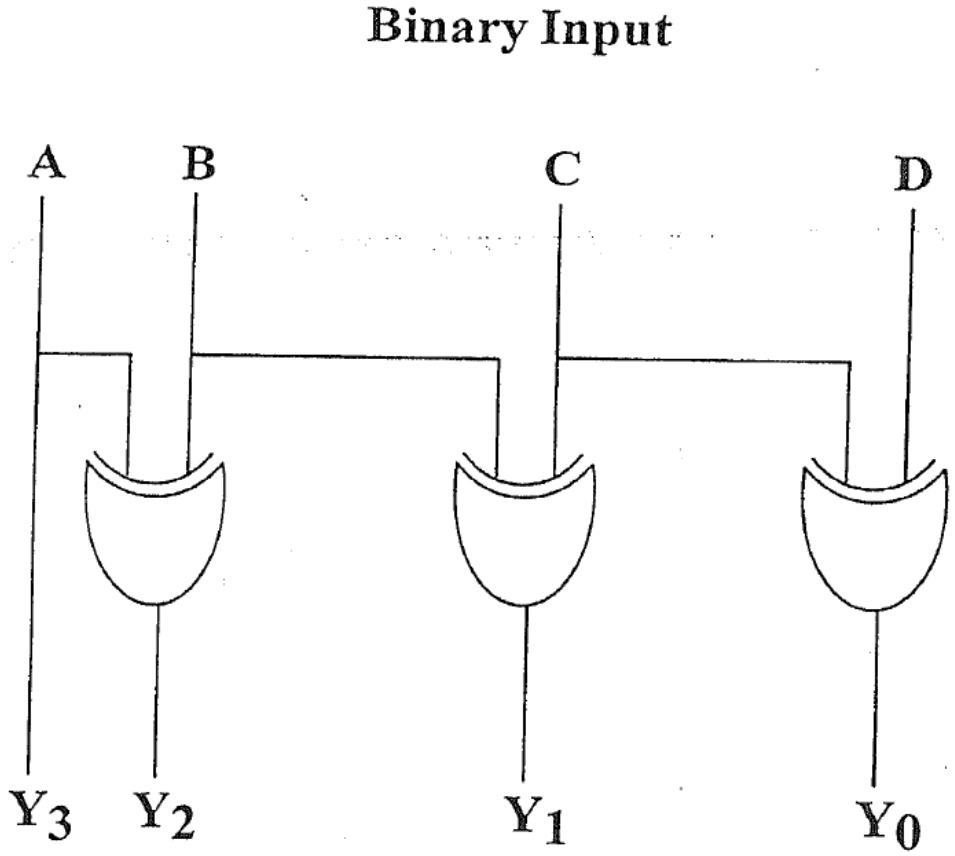
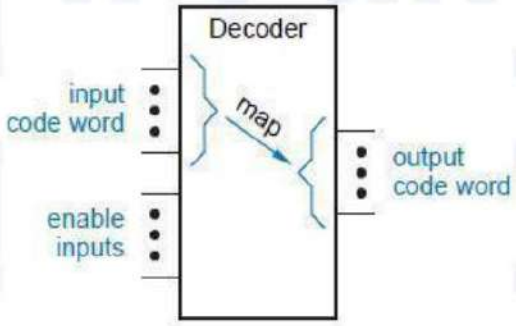


Figure 3.1 Binary-to-Gray Code

Decoders & Encoders

Decoders

A *decoder* is a multiple-input, multiple-output logic circuit that converts coded inputs into coded outputs, where the input and output codes are different. The input code generally has fewer bits than the output code, and there is a one-to one mapping from input code words into output code words. In a *one-to-one mapping*, each input code word produces a different output code word.



The general structure of a decoder circuit is shown in Figure 1. The enable inputs, if present, must be asserted for the decoder to perform its normal mapping function. Otherwise, the decoder maps all input code words into a single, "disabled," output code word.

The most commonly used output code is a 1-out-of-m code, which contains m bits, where one bit is asserted at any time. Thus, in a 1-out-of-4 code with active-high outputs, the code words are 0001, 0010, 0100, and 1000. With active-low outputs, the code words are 1110, 1101, 1011, and 0111.

Binary Decoders

The most common decoder circuit is an n-to-$2n$ decoder or *binary decoder*. Such a decoder has an n-bit binary input code and a 1-out-of-$2n$ output code. A binary decoder is used when you need to activate exactly one of $2n$ outputs based on an n-bit input value.

Inputs			*Outputs*			
EN	I1	I0	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Table 1: 2 to 4 decoder

Table 1 is the truth table of a 2-to-4 decoder. The input code word 1,I0 represents an integer in the range 0–3. The output code word Y3,Y2,Y1,Y0 has Y*i* equal to 1 if and only if the input code word is the binary representation of *i* and the *enable input* EN is 1. If EN is 0, then all of the outputs are 0. A gate-level circuit for the 2-to-4 decoder is shown in Figure 2 Each AND gate *decodes* one combination of the input code word I1,I0.

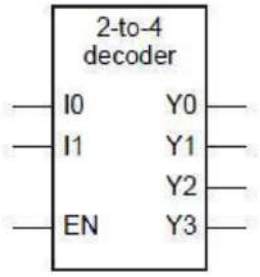


Fig 3: 2 to 4 decoder logic symbol

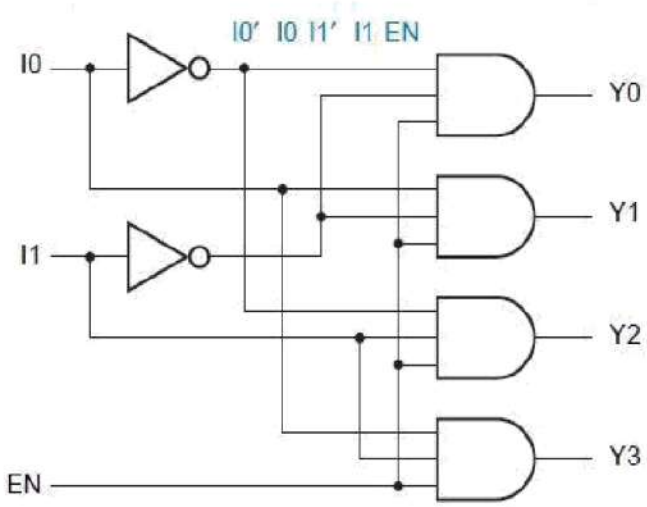


Fig 4: logic diagram of 2 to 4 decoder

The 74x139 Dual 2-to-4 Decoder

Two independent and identical 2-to-4 decoders are contained in a single MSI part, the *74x139*. The gate-level circuit diagram for this IC is shown in Figure 5.

1. The outputs and the enable input of the '139 are active-low.
2. Most MSI decoders were originally designed with active-low outputs, since TTL inverting gates are generally faster than non inverting ones.

3. '139 has extra inverters on its select inputs. Without these inverters, each select input would present three AC or DC loads instead of one, consuming much more of the fanout budget of the device that drives it.

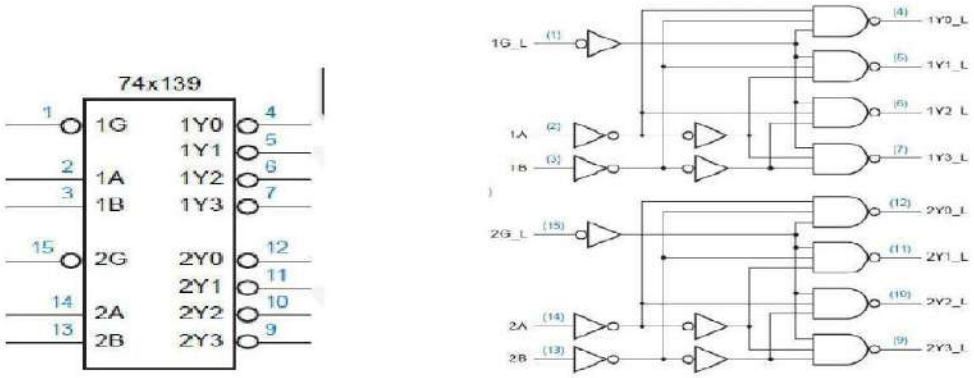


Figure 6 The 74x139 dual 2-to-4 decoder: (a) traditional logic symbol (b) logic diagram, including pin numbers for a standard 16-pin dual in-line package;

In this case, the assignment of the generic function to one half or the other of a particular '139 package can be deferred until the schematic is completed Table 5-6 is the truth table for a 74x139-type decoder.

The 74x138 3-to-8 Decoder

The *74x138* is a commercially available MSI 3-to-8 decoder whose gate-level circuit diagram and symbol are shown in Figure 7; its truth table is given in Table 5-7. Like the 74x139, the 74x138 has active-low outputs, and it has three enable inputs (G1, /G2A, /G2B), all of which must be asserted for the selected output to be asserted.

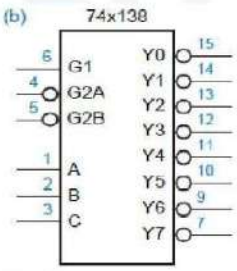


Figure 7: logic symbol of 74X138

Inputs						Outputs							
G1	G2A_L	G2B_L	C	B	A	Y7_L	Y6_L	Y5_L	Y4_L	Y3_L	Y2_L	Y1_L	Y0_L
0	x	x	x	x	x	1	1	1	1	1	1	1	1
x	1	x	x	x	x	1	1	1	1	1	1	1	1
x	x	1	x	x	x	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	0	0	1	1	1	0	1	1	1	1
1	0	0	1	0	1	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1

DO NOT COPY

The logic function of the '138 is straightforward—an output is asserted if and only if the decoder is enabled and the output is selected. Thus, we can easily write logic equations for an internal output signal such as Y5 in terms of the internal input signals:

$$Y5 = \underbrace{G1 \cdot G2A \cdot G2B}_{\text{enable}} \cdot \underbrace{C \cdot B' \cdot A}_{\text{select}}$$

However, because of the inversion bubbles, we have the following relations between internal and external signals:

$$G2A = G2A_L'$$
$$G2B = G2B_L'$$
$$Y5 = Y5_L'$$

Therefore, if we're interested, we can write the following equation for the external output signal Y5_L in terms of external input signals:

$$\begin{aligned} Y5_L = Y5' &= (G1 \cdot G2A_L' \cdot G2B_L' \cdot C \cdot B' \cdot A)' \\ &= G1' + G2A_L + G2B_L + C' + B + A' \end{aligned}$$

On the surface, this equation doesn't resemble what you might expect for a decoder, since it is a logical sum rather than a product. However, if you practice bubble-to-bubble logic design, you don't have to worry about this; you just give the output signal an active-low name and remember that it's active low when you connect it to other inputs.

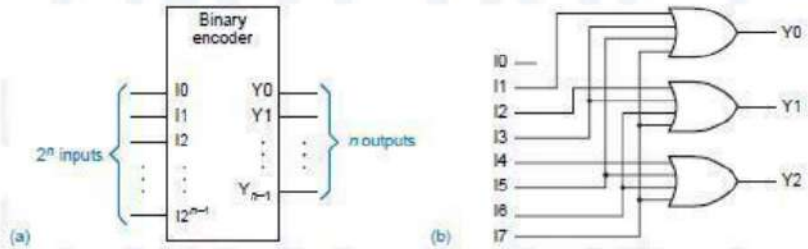
5.5 Encoders

A decoder's output code normally has more bits than its input code. If the device's output code has *fewer* bits than the input code, the device is usually called an *encoder*.

Probably the simplest encoder to build is a $2n$-to-n or *binary encoder*. As shown in Figure 5-45(a), it has just the opposite function as a binary *de*coder— its input code is the 1-out-of-$2n$ code and its output code is n-bit binary. The equations for an 8-to-3 encoder with inputs I0–I7 and outputs Y0–Y2 are given below:

$$Y0 = I1 + I3 + I5 + I7$$
$$Y1 = I2 + I3 + I6 + I7$$
$$Y2 = I4 + I5 + I6 + I7$$

The corresponding logic circuit is shown in (b). In general, a $2n$-to-n encoder can be built from n $2n$ -input1 OR gates. Bit i of the input code is connected to OR gate j if bit j in the binary representation of i is 1.



Priority Encoder:

The 1-out-of-2*n* coded outputs of an *n*-bit binary decoder are generally used to control a set of 2*n* devices, where at most one device is supposed to be active at any time. Conversely, consider a system with 2*n inputs*, each of which indicates a request for service. This structure is often found in microprocessor input/output subsystems, where the inputs might be interrupt requests.

In this situation, it may seem natural to use a binary encoder. to observe the inputs and indicate which one is requesting service at any time. However, this encoder works properly only if the inputs are guaranteed to be asserted at most one at a time. If multiple requests can be made simultaneously, the encoder gives undesirable results. For example, suppose that inputs I2 and I4 of the 8-to-3 encoder are both 1; then the output is 110, the binary encoding of 6.

Y0 = I1 + I3 + I5 + I7
Y1 = I2 + I3 + I6 + I7
Y2 = I4 + I5 + I6 + I7

Either 2 or 4, not 6, would be a useful output in the preceding example, but how can the encoding device decide which? The solution is to assign *priority* to the input lines, so that when multiple requests are asserted, the encoding device produces the number of the highest-priority requestor. Such a device is called a *priority encoder*.

Input I7 has the highest priority. Outputs A2–A0 contain the number of the highest-priority asserted input, if any. The IDLE output is asserted if no inputs are asserted.

In order to write logic equations for the priority encoder's outputs, we first define eight intermediate variables H0–H7, such that Hi is 1 if and only if Ii is the highest priority 1 input: Using these signals, the equations for the A2–A0 outputs are similar to the ones for a simple binary encoder:

H7=I7 (Highest Priority) H6=I6.I7'

H5=I5.I6'.I7'
H4=I4.I5'.I6'.I7'
H3=I3.I4'.I5'.I6'.I7'
H2=I2.I3'.I4'.I5'.I6'.I7'
H1=I1. I2'.I3'.I4'.I5'.I6'.I7'
H0=I0.I1'. I2'.I3'.I4'.I5'.I6'.I7'
IDLE= I0'.I1'. I2'.I3'.I4'.I5'.I6'.I7'

Encoder

A0=Y0 = H1 + H3 + H5 + H7
A1=Y1 = H2 + H3 + H6 + H7
A2=Y2 = H4 + H5 + H6 + H7

Multiplexers:

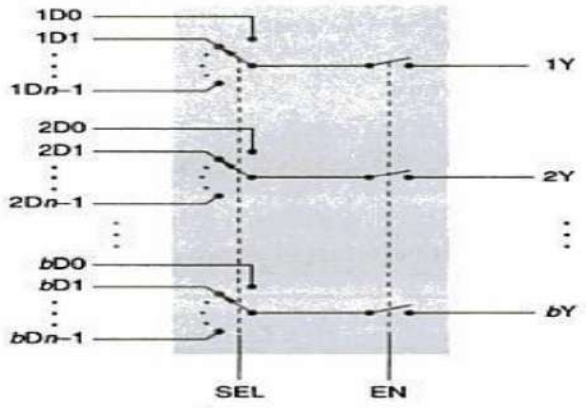
A *multiplexer* is a digital switch—it connects data from one of n sources to its output. Figure 5-61(a) shows the inputs and outputs of an n-input, b-bit multiplexer. There are n sources of data, each of which is b bits wide. A multiplexer is often called a *mux* for short.

A multiplexer can use addressing bits to select one of several input bits to be the output. A selector chooses a single data input and passes it to the MUX output.It has one output selected at a time.

Figure shows a switch circuit that is roughly equivalent to the multiplexer. However, unlike a mechanical switch, a multiplexer is a unidirectional device: information flows only from inputs (on the left) to outputs (on the right. Multiplexers are obviously useful devices in any application in which data must be switched from multiple sources to a destination. A common application

in computers is the multiplexer between the processor's registers and its arithmetic logic unit (ALU). For example, consider a 16-bit processor in which each instruction has a 3-bit field that specifies one of eight registers to use. This 3-bit field is connected to the select inputs of an 8-input, 16-bit multiplexer. The multiplexer's data inputs are connected to the eight registers, and its data outputs

are connected to the ALU to execute the instruction using the selected register.



The sizes of commercially available MSI multiplexers are limited by the number of pins available in an inexpensive IC package. Commonly used muxes come in 16-pin packages. Shown in fig which selects among eight 1-bit inputs. The select inputs are named C, B, and A, where C is most significant numerically. The enable input EN_L is active low; both active-high (Y) and active-low (Y_L) versions of the output are provided.

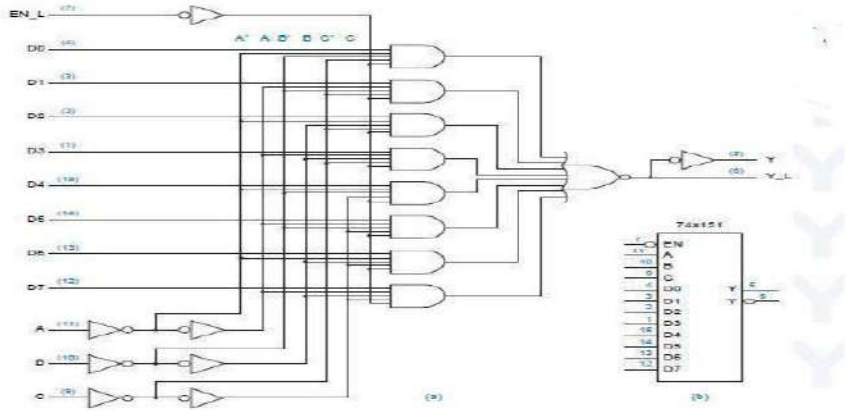
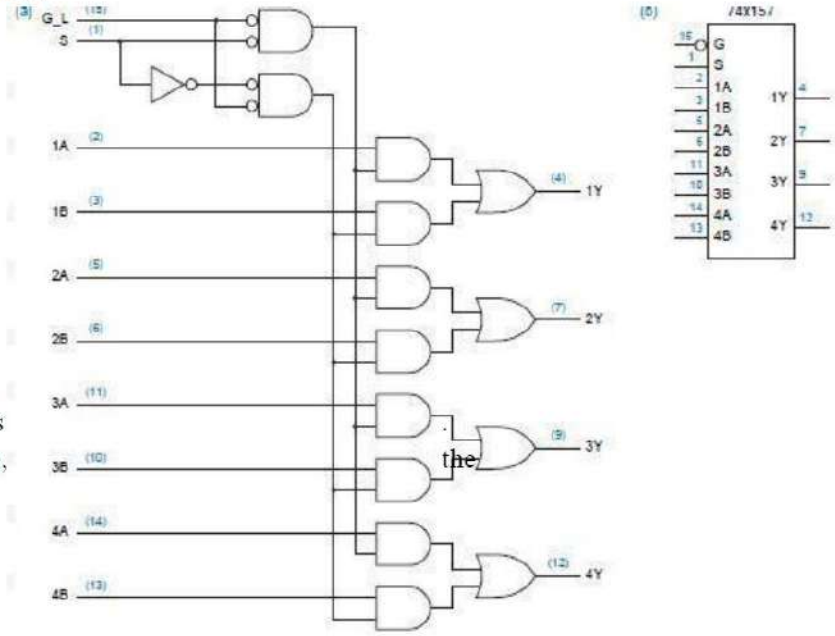


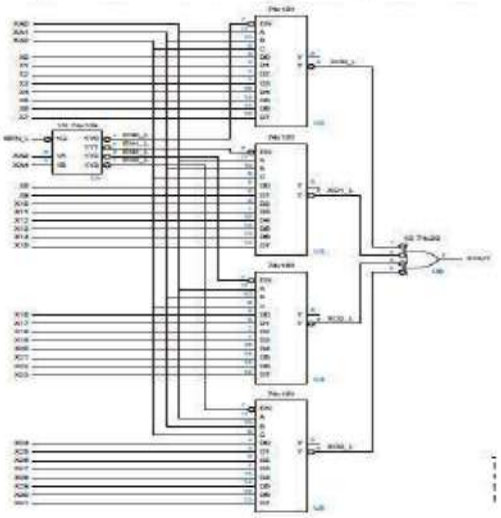
Figure The 74x151 8-input, 1-bit multiplexer: (a) logic diagram, including pin numbers for a standard 16-pin dual in-line
package; (b) traditional logic symbol.

At the other extreme of muxes in 16-pin packages, we have the *74x157*, shown in Figure, which selects between two 4-bit inputs. Just to confuse things, the manufacturer has named the select input S and the active-low enable input G_L. Also note that the data sources are named A and B .



Expanding
Seldom does
For example,
the

design of a computer processor. This function could be performed by 16 74x151 8-input, 1-bit multiplexers or equivalent ASIC cells, each handling one bit of all the inputs and the output. The processor's 3-bit register-select field would be connected to the A, B, and C inputs of all 16 muxes, so they would all select the same register source at any given time.



.

Another dimension in which multiplexers can be expanded is the number of data sources. For example, suppose we needed a 32-input, 1-bit multiplexer. Figure shows one way to build it. Five select bits are required. A 2-to-4 decoder (one-half of a 74x139) decodes the two high-order select bits to enable one of four 74x151 8-input multiplexers. Since only one

'151 is enabled at a time, the '151 outputs can simply be ORed to obtain the final output.

The 32-to-1 multiplexer can also be built using 74x251s. The circuit is

identical to Figure 5-65, except that the output NAND gate is eliminated. Instead, the Y (and, if desired, Y_L) outputs of the four '251s are simply tied together. The '139 decoder ensures that at most one of the '251s has its threestate outputs enabled at any time. If the '139 is disabled (XEN_L is negated), then all of the '251s are disabled, and the XOUT and XOUT_L outputs are

undefined. However, if desired, resistors may be connected from each of these signals to 5 volts to pull the output HIGH in this case.

Multiplexers, Demultiplexers, and Buses

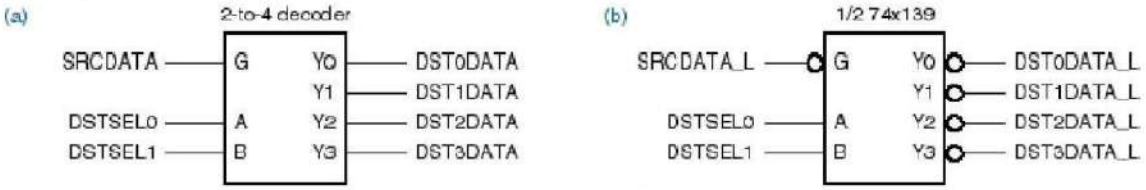
A multiplexer can be used to select one of *n* sources of data to transmit on a bus. At the far end of the bus, a *demultiplexer* can be used to route the bus data to one of *m* destinations. Such an application, using a 1-bit bus. In fact, block diagrams for logic circuits often depict multiplexers and demultiplexers , to suggest visually how a selected one of multiple data sources gets directed onto a bus and routed to a selected one of multiple destinations.

The function of a demultiplexer is just the inverse of a multiplexer's. For example, a 1-bit, *n*-output demultiplexer has one data input and *s* inputs to select one of *n s* 2data outputs. In normal operation, all outputs except the selected one are 0; the selected output equals the data input. This definition may be generalized for a *b*-bit, *n*-output demultiplexer; such a device has *b* data inputs, and its *s* select inputs choose one of *n s* 2sets of *b* data outputs.

A binary decoder with an enable input can be used as a demultiplexer, as shown in Figure 5-67. The decoder's enable input is connected to the data line, and its select inputs determine which of its output lines is driven with the data bit. The remaining output lines are negated.

Thus, the 74x139 can be used as a 2-bit, 4-output demultiplexer with active-low data inputs and outputs, and the
74x138 can be used as a 1-bit, 8-output demultiplexer. In fact, the manufacturer's catalog typically lists these ICs as "decoders/demultiplexers."

A Mux is used to select one of n sources of data to transmit on a bus. A demultiplexer can be used to route the bus data to one of m destinations. Just the inverse of a mux. A binary decoder with an enable input can be used as a Demux. E.g. 74x139 can be used as a 2-bit, 4-output Demux.



Copyright © 2000 by Prentice Hall, Inc.
Digital Design Principles and Practices, 3/e

DO NOT

Parity Generators/Checkers:

Parity Circuits

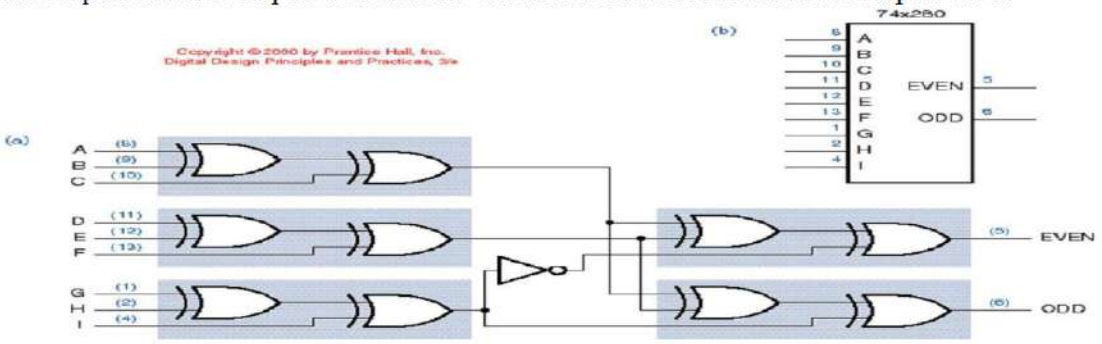
N XOR gates may be cascaded to form a circuit with *n* 1 inputs and a single output. This is called an *odd-parity circuit*, because its output is 1 if an odd number of its inputs are 1.

If the output of either circuit is inverted, we get an *even-parity circuit*, whose output is 1 if an even number of its inputs are 1.

The 74x280 9-Bit Parity Generator

Rather than build a multibit parity circuit with discrete XOR gates, it is more economical to put all of the XORs in a single MSI package with just the primary inputs and outputs available at the external pins. The *74x280* 9-bit parity generator, shown in Figure is such a device. It has nine inputs and two outputs that indicate whether an even or odd number of inputs are 1.

Copyright ©2000 by Prentice Hall, Inc.
Digital Design Principles and Practices, 3/e



Parity-Checking Applications

described error-detecting codes that use an extra bit, called a parity bit, to detect errors in the transmission and storage of data. In an evenparity code, the parity bit is chosen so that the total number of 1 bits in a code word is even. Parity circuits like the 74x280 are used both to generate the correct value of the parity bit when a code word is stored or transmitted, and to check the parity bit when a code word is retrieved or received.

Figure shows how a parity circuit might be used to detect errors in the memory of a microprocessor system. The memory stores 8-bit bytes, plus a parity bit for each byte. The microprocessor uses a bidirectional bus D[0:7] to transfer data to and from the memory. Two control lines, RD and WR, are used to indicate whether a read or write operation is desired, and an ERROR signal is asserted to indicate parity errors during read operations. Complete details of the memory chips, such as addressing inputs, are not shown; memory chips are described in detail in \chapref{MEMORY}.

Parallel Binary Adder/ Subtractor

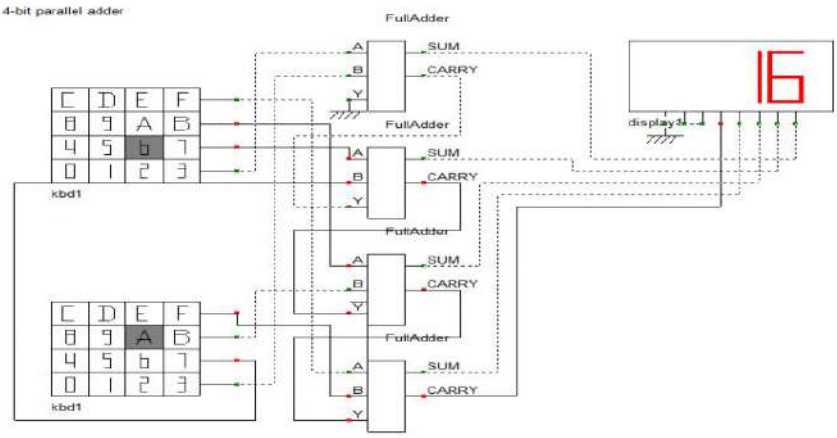
A parallel adder is an arithmetic combinational logic circuit that is used to add more than one bit of data simultaneously. A full adder adds two bits and a carry to give an output. However, to add more than one bit of data in length a parallel adder is used. A parallel adder adds corresponding bits simultaneously using full adders and keeps generating a carry and pushing it towards the next most significant bit to be added. An n-bit parallel adder uses n full adders connected in cascade with each full adder adding the two corresponding bits of both the numbers.

For example, for a binary number D3D2D1D0 and B3B2B1B0, a full adder connected in cascade would add D0 and B0 and send the result to be displayed (LSB). If a carry is generated, it will be passed on to the input of the next full adder.

How to design a 4-bit parallel adder?

To add two Hex codes we need four full adders connected in cascade. This is because a hex code can be represented by four binary bits. The four full adders will connect to each other via their CARRY outputs. And depending on the position of the bits the full adders add, the SUM outputs of the full adders will be connected to the display. The least significant bit will be connected to the LSB of the display. The most significant bit will be connected to the pin one bit before the MSB of the display. The carry output of the final full adder will be connected to the MSB pin of the display.

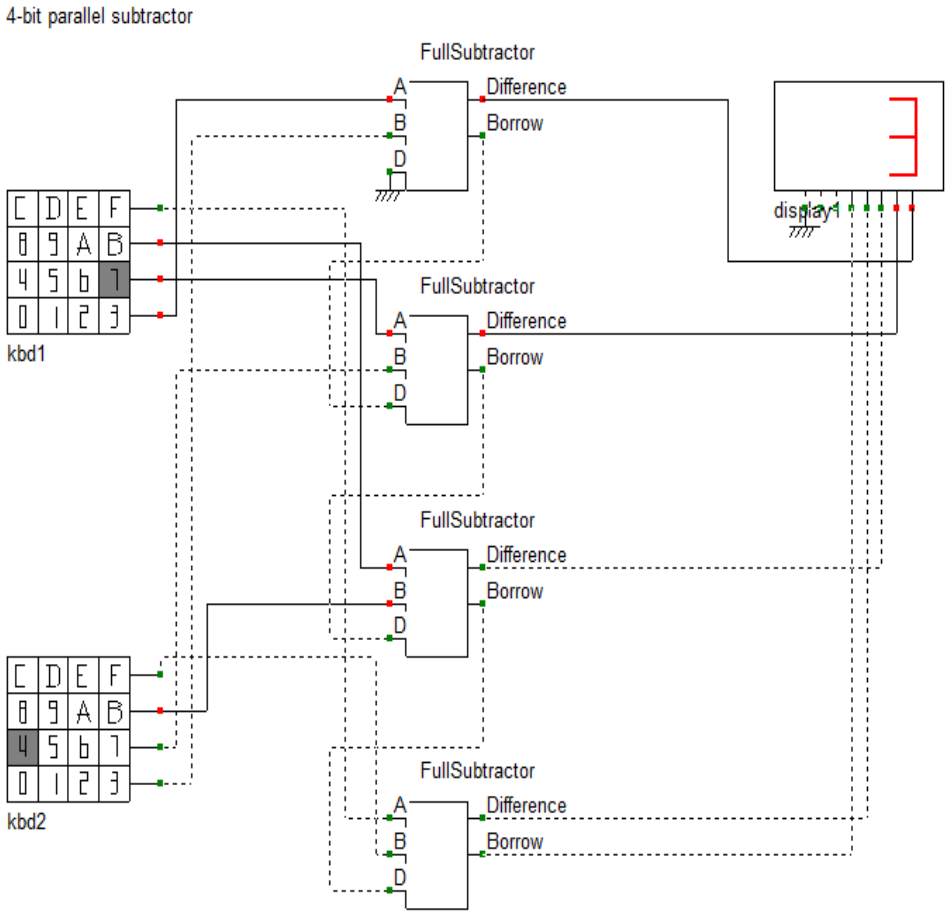
We can use two hex 4×4 keypads to generate the input bits or we can just add the bits manually. If you want a circuit diagram without the hex keypads as inputs let us know in the comments. Each row of the keypad is connected to a full adder depending on its significance. The first full adder receives inputs from the first row of the hex keypad, the second receives inputs from the second row of the hex keypad and the carry from the first and so on. The resultant combinational logic circuit is shown below.



How to design a 4-bit parallel subtractor?

A 4-bit parallel subtractor is used to subtract a number consisting of 4 bits. This is done by cascading a series of full subtractors. For an n-bit parallel subtractor, n full subtractors can be cascaded to achieve the desired output. The connections are exactly the same as that of the 4-bit parallel adder which we saw earlier in this post. Each of the bit is subtracted from its corresponding bit of equal significance from the other number. A borrow if generated, propagates through the cascade of full subtractors.

We use the same 4×4 hex keypads to input data in a full subtractor. The first rows (having the least significance compared to the other rows) of the hex keypads are connected to the first full subtractor. The second rows to the second full subtractor's inputs along with the borrow from the first full subtractor and so on. The output of each full adder is connected to the display on the basis of their positional significance in the answer. i.e the output of the first full subtractor is connected to the LSB of the display. The final borrow output of the final full subtractor is connected to the MSB pin of the display



Magnitude Comparators:

Comparators

Comparing two binary words for equality is a commonly used operation in computer systems and device interfaces. we showed a system structure in which devices are enabled by comparing a "device select" word with a predetermined "device ID." A circuit that compares two binary words and indicates whether they are equal is called a *comparator*. Some comparators interpret their input words as signed or unsigned numbers and also indicate an arithmetic relationship (greater or less than) between the words. These devices are often called *magnitude comparators*.

Inputs				Outputs		
A_1	A_0	B_1	B_0	A > B	A = B	A< B
0	0	0	0	0	1	0
0	0	0	1	0	0	1
0	0	1	0	0	0	1
0	0	1	1	0	0	1
0	1	0	0	1	0	0
0	1	0	1	0	1	0
0	1	1	0	0	0	1
0	1	1	1	0	0	1
1	0	0	0	1	0	0
1	0	0	1	1	0	0
1	0	1	0	0	1	0
1	0	1	1	0	0	1
1	1	0	0	1	0	0
1	1	0	1	1	0	0
1	1	1	0	1	0	0
1	1	1	1	0	1	0

A > B

A_1A_0 \ B_1B_0	00	01	11	10
00	0	0	0	0
01	1	0	0	0
11	1	1	0	1
10	1	1	0	0

$$A>B = A_0\overline{B}_1\overline{B}_0 + A_1\overline{B}_1 + A_1A_0\overline{B}_0$$

A = B

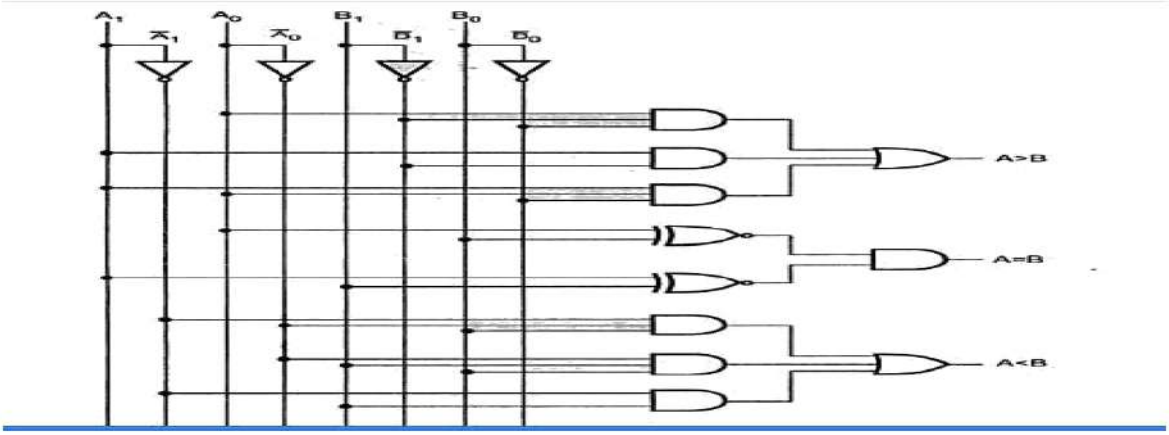
A_1A_0 \ B_1B_0	00	01	11	10
00	1	0	0	0
01	0	1	0	0
11	0	0	1	0
10	0	0	0	1

A < B

A_1A_0 \ B_1B_0	00	01	11	10
00	0	1	1	1
01	0	0	1	1
11	0	0	0	0
10	0	0	1	0

Fig. 4.93

$$\begin{aligned}(A = B) &= \overline{A}_1\overline{A}_0\overline{B}_1\overline{B}_0 + \overline{A}_1A_0\overline{B}_1B_0 + A_1A_0B_1B_0 + A_1\overline{A}_0B_1\overline{B}_0 \\ &= \overline{A}_1\overline{B}_1(\overline{A}_0\overline{B}_0 + A_0B_0) + A_1B_1(A_0B_0 + \overline{A}_0\overline{B}_0) \\ &= (A_0 \odot B_0)(A_1 \odot B_1) \\ (A < B) &= \overline{A}_1\overline{A}_0B_0 + \overline{A}_0B_1B_0 + \overline{A}_1B_1\end{aligned}$$


A1
A0
B1
B0
A>B
A=B
A<B

UNIT-V

SEQUENTIAL LOGIC IC'S AND MEMORIES

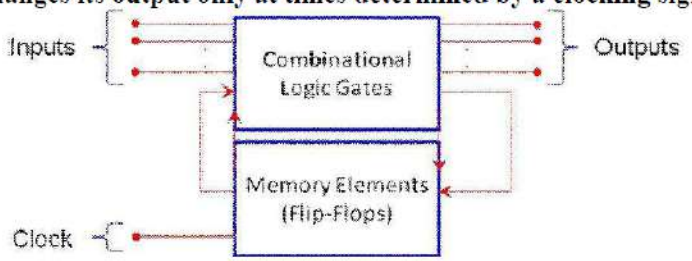
All Types of Flip-flops:

A *bistable memory device* is the generic term for the elements we are studying.

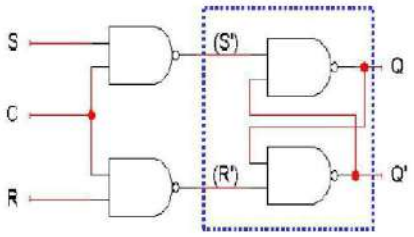
Latches and Flip flops

***Latches and flip-flops* (FFs) are the basic building blocks of sequential circuits.**

latch: bistable memory device with level sensitive triggering (no clock), watches all of its inputs continuously and changes its outputs at any time, independent of a clocking signal.
flip-flop: bistable memory device with edge-triggering (with clock), samples its inputs, and changes its output only at times determined by a clocking signal.



- Here is an SR latch with a control input C.
- Notice the hierarchical design!
  - The dotted blue box is the S'R' latch.
  - The additional NAND gates are simply used to generate the correct inputs for the S'R' latch.
- The control input acts just like an enable.



C	S	R	S'	R'	Q
0	x	x	1	1	No change
1	0	0	1	1	No change
1	0	1	1	0	0 (reset)
1	1	0	0	1	1 (set)
1	1	1	0	0	Avoid!

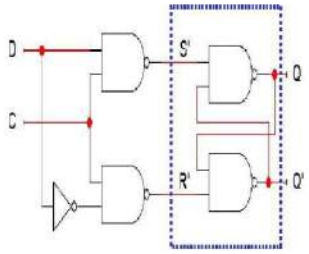
D latch

- Finally, a D latch is based on an S'R' latch. The additional gates generate the S' and R' signals, based on inputs D ("data") and C ("control").

– When C = 0, S' and R' are both 1, so the state Q does not change.

– When C = 1, the latch output Q will equal the input D.

- No more messing with one input for set and another input for reset!

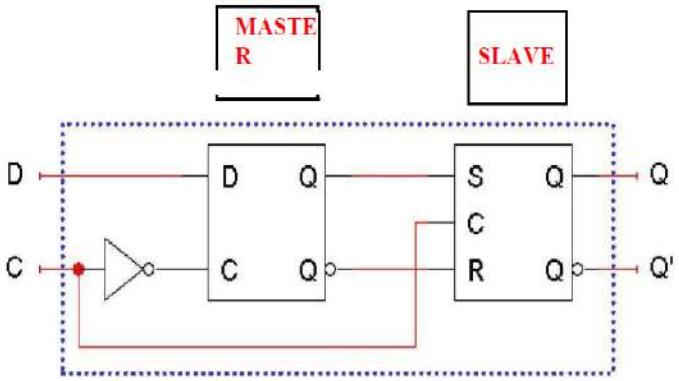


C	D	Q
0	x	No change
1	0	0
1	1	1

- Also, this latch has no "bad" input combinations to avoid. Any of the four possible assignments to C and D are valid.

Flip-flops

- Here is the internal structure of a D flip-flop.
  - The flip-flop inputs are C and D, and the outputs are Q and Q'.
  - The D latch on the left is the master, while the SR latch on the right is called the slave.
- Note the layout here.
  - The flip-flop input D is connected directly to the master latch.
  - The master latch output goes to the slave.
  - The flip-flop outputs come directly from the slave latch.



D flip-flops when C=0

- The D flip-flop's control input C enables *either* the D latch or the SR latch, but not both.
- When C = 0:
  - The master latch is enabled, and it monitors the flip-flop input D. Whenever D changes, the master's output changes too.
  - The slave is disabled, so the D latch output has no effect on it. Thus, the slave just maintains the flip-flop's current state.

D flip-flops when C=1

- *As soon as C becomes 1,*
  - The master is disabled. Its output will be the *last* D input value seen just before C became 1.
  - Any subsequent changes to the D input while C = 1 have no effect on the master latch, which is now disabled.
  - The slave latch is enabled. Its state changes to reflect the master's output, which again is the D input value from right when C became 1.

Positive edge triggering

- This is called a positive edge-triggered flip-flop.
  - The flip-flop output Q changes *only* after the positive edge of C.
  - The change is based on the flip-flop input values that were present right at the positive edge of the clock signal.

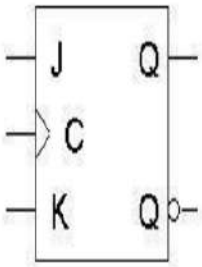
The D flip-flop's behavior is similar to that of a D latch except for the positive edge-triggered nature, which is not explicit in this table

C	D	Q
0	x	No change
1	0	0 (reset)
1	1	1 (set)

Flip-flop variations

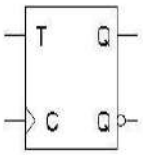
- We can make different versions of flip-flops based on the D flip-flop, just like we made different latches based on the S'R' latch.

- A JK flip-flop has inputs that act like S and R, but the inputs JK=11 are used to *complement* the flip-flop's current state.



C	J	K	Qnext
0	x	x	No change
1	0	0	No change
1	0	1	0 (reset)
1	1	0	1 (set)
1	1	1	Q'current

A T flip-flop can only maintain or complement its current state



C	T	Qnext
0	x	No change
1	0	No change
1	1	Q'current

Characteristic equations

- We can also write characteristic equations, where the next state Q(t+1) is defined in terms of the current state Q(t) and inputs.

D	Q(t+1)	Operation
0	0	Reset
1	1	Set

Q(t+1) = D

J	K	Q(t+1)	Operation
0	0	Q(t)	No change
0	1	0	Reset
1	0	1	Set
1	1	Q'(t)	Complement

Q(t+1) = K'Q(t) + JQ'(t)

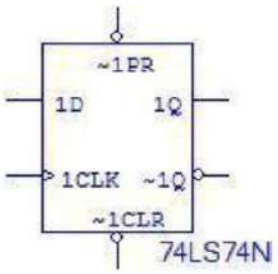
T	Q(t+1)	Operation
0	Q(t)	No change
1	Q'(t)	Complement

Q(t+1) = T'Q(t) + TQ'(t) = T Q(t)

Flip-Flop Vs. Latch

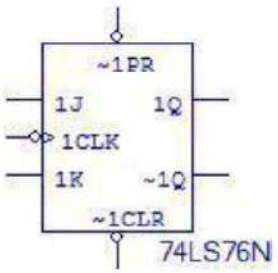
- The primary difference between a D flip-flop and D latch is the EN/CLOCK input.
- The flip-flop's CLOCK input is edge sensitive, meaning the flip-flop's output changes on the edge (rising or falling) of the CLOCK input.
- The latch's EN input is level sensitive, meaning the latch's output changes on the level (high or low) of the EN input.

Flip-Flops & Latches



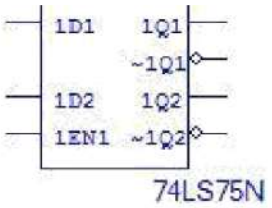
74LS74

Dual Positive-Edge-Triggered D Flip-Flops with Preset, Clear, and Complementary Outputs



74LS76

Dual Negative-Edge-Triggered J-K Flip-Flops with Preset, Clear, and Complementary Outputs



74LS75 Quad Latch

74LS74: D Flip-Flop

Function Table

Inputs				Outputs	
PR	CLR	CLK	D	Q	$\overline{Q}$
L	H	X	X	H	L
H	L	X	X	L	H
L	L	X	X	H (Note 1)	H (Note 1)
H	H	↑	H	H	L
H	H	↑	L	L	H
H	H	L	X	Q_0	$\overline{Q}_0$

H = HIGH Logic Level
X = Either LOW or HIGH Logic Level
L = LOW Logic Level
↑ = Positive-going Transition
Q_0 = The output logic level of Q before the indicated input conditions were established.

Note 1: This configuration is nonstable; that is, it will not persist when either the preset and/or clear inputs return to their inactive (HIGH) level.

74LS76: J/K Flip-Flop

Function Table

Inputs					Outputs	
PR	CLR	CLK	J	K	Q	$\overline{Q}$
L	H	X	X	X	H	L
H	L	X	X	X	L	H
L	L	X	X	X	H (Note 1)	H (Note 1)
H	H	⎍	L	L	Q_o	$\overline{Q}_o$
H	H	⎍	H	L	H	L
H	H	⎍	L	H	L	H
H	H	⎍	H	H	Toggle	

H = High Logic Level
L = Low Logic Level
X = Either Low or High Logic Level
⎍ = Positive pulse data. The J and K inputs must be held constant while the clock is high. Data is transfered to the outputs on the falling edge of the clock pulse.
Q_0 = The output logic level before the indicated input conditions were established.
Toggle = Each output changes to the complement of its previous level on each complete active high level clock pulse.
Note 1: This configuration is nonstable; that is, it will not persist when the preset and/or clear inputs return to their inactive (high) level.

Conversion of Flip flops

For the conversion of one **flip flop** to another, a combinational circuit has to be designed first. If a JK Flip Flop is required, the inputs are given to the combinational circuit and the output of the combinational circuit is connected to the inputs of the actual flip flop. Thus, the output of the actual flip flop is the output of the required flip flop. In this post, the following flip flop conversions will be explained.

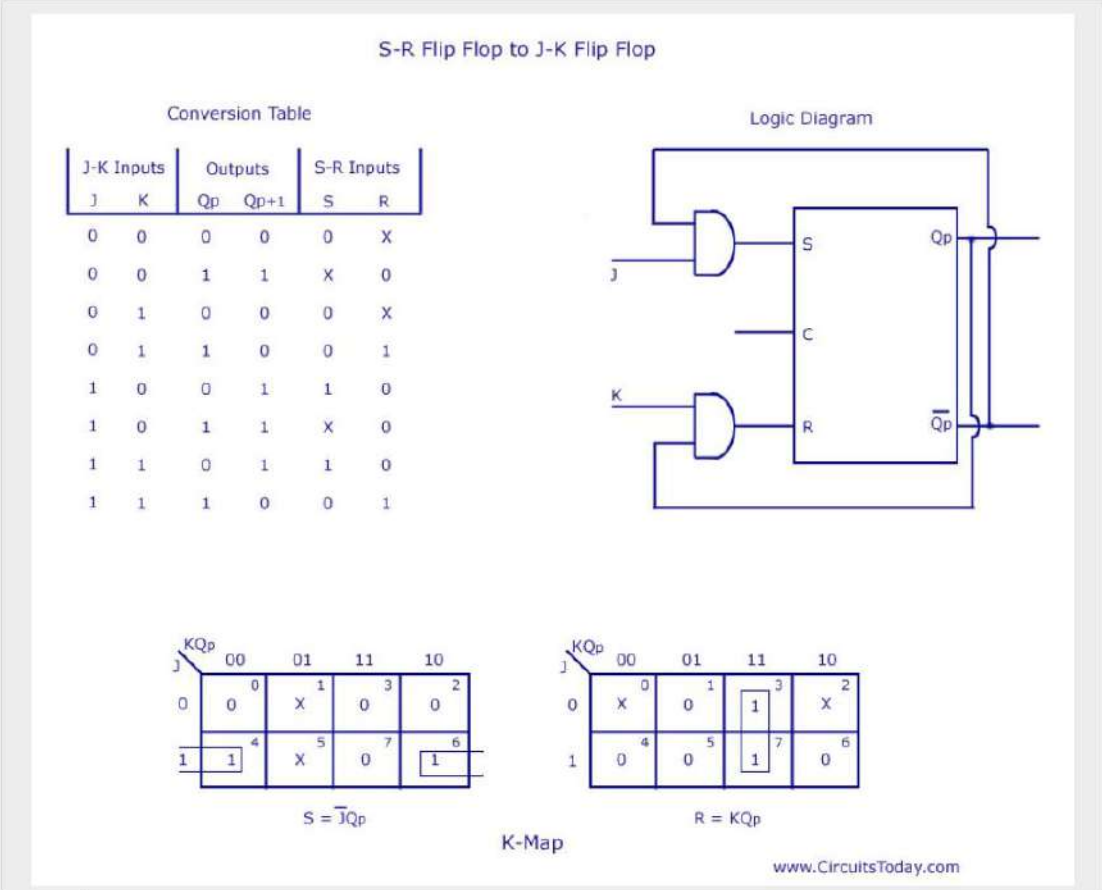
- **SR Flip Flop to JK Flip Flop**
- **JK Flip Flop to SR Flip Flop**
- **SR Flip Flop to D Flip Flop**
- **D Flip Flop to SR Flip Flop**
- **JK Flip Flop to T Flip Flop**
- **JK Flip Flop to D Flip Flop**
- **D Flip Flop to JK Flip Flop**

SR Flip Flop to JK Flip Flop

As told earlier, J and K will be given as external inputs to S and R. As shown in the logic diagram below, S and R will be the outputs of the combinational circuit.

The truth tables for the flip flop conversion are given below. The present state is represented by Qp and Qp+1 is the next state to be obtained when the J and K inputs are applied.

For two inputs J and K, there will be eight possible combinations. For each combination of J, K and Qp, the corresponding Qp+1 states are found. Qp+1 simply suggests the future values to be obtained by the JK flip flop after the value of Qp. The table is then completed by writing the values of S and R required to get each Qp+1 from the corresponding Qp. That is, the values of S and R that are required to change the state of the flip flop from Qp to Qp+1 are written.

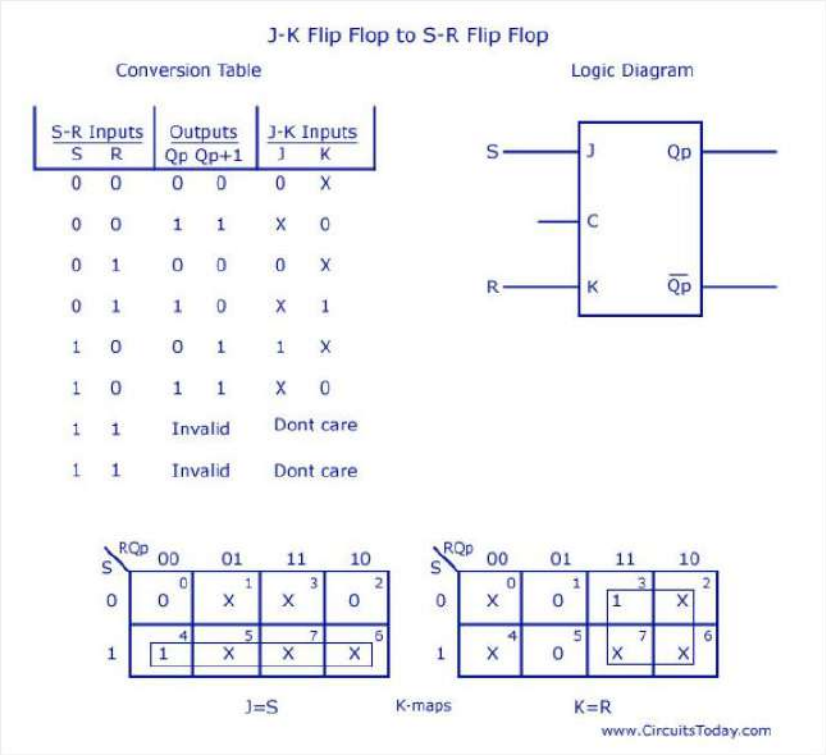


SR Flip Flop to JK Flip Flop

⊡ **JK Flip Flop to SR Flip Flop**

This will be the reverse process of the above explained conversion. S and R will be the external inputs to J and K. As shown in the logic diagram below, J and K will be the outputs of the combinational circuit. Thus, the values of J and K have to be obtained in terms of S, R and Qp. The logic diagram is shown below.

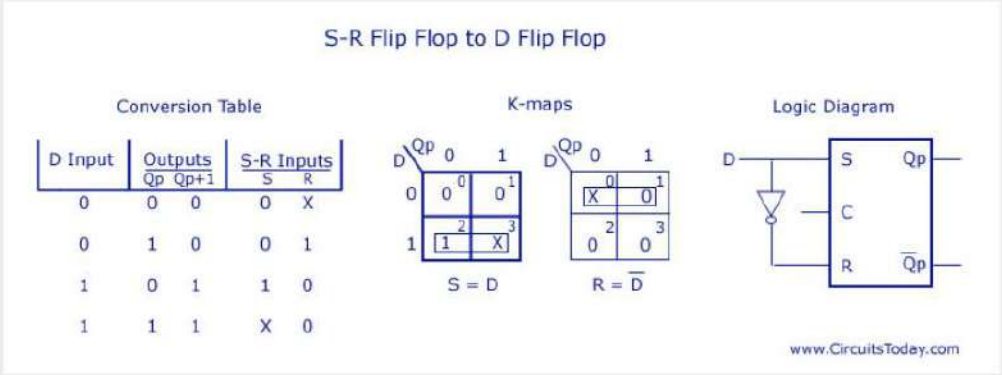
A conversion table is to be written using S, R, Qp, Qp+1, J and K. For two inputs, S and R, eight combinations are made. For each combination, the corresponding Qp+1 outputs are found ut. The outputs for the combinations of S=1 and R=1 are not permitted for an SR flip flop. Thus the outputs are considered invalid and the J and K values are taken as "don't cares".



JK Flip Flop to SR Flip Flop

⊡ SR Flip Flop to D Flip Flop

As shown in the figure, S and R are the actual inputs of the flip flop and D is the external input of the flip flop. The four combinations, the logic diagram, conversion table, and the K-map for S and R in terms of D and Qp are shown below.

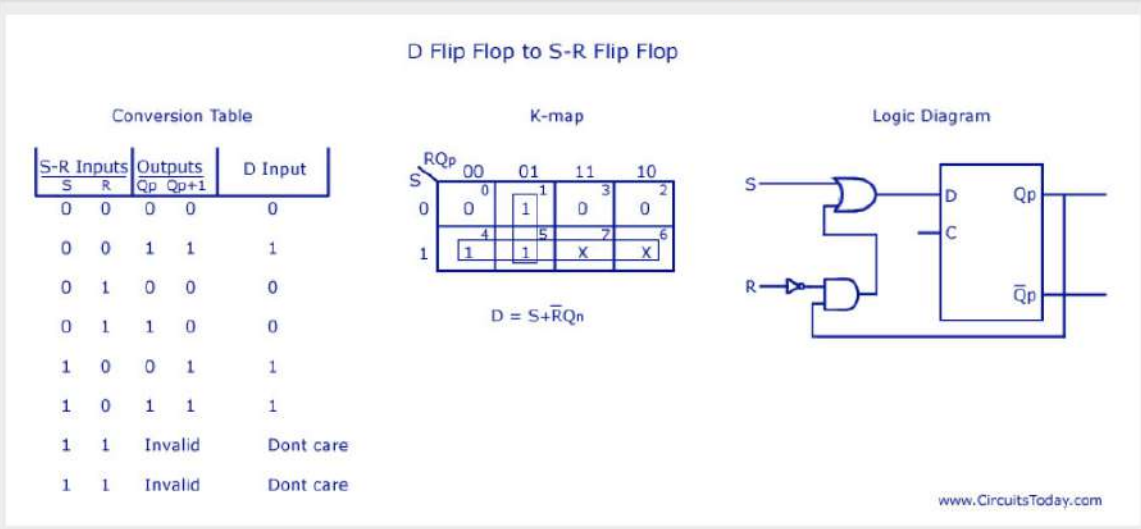


SR Flip Flop to D Flip Flop

⊡ D Flip Flop to SR Flip Flop

D is the actual input of the flip flop and S and R are the external inputs. Eight possible combinations are achieved from the external inputs S, R and Qp. But, since the combination of S=1 and R=1 are

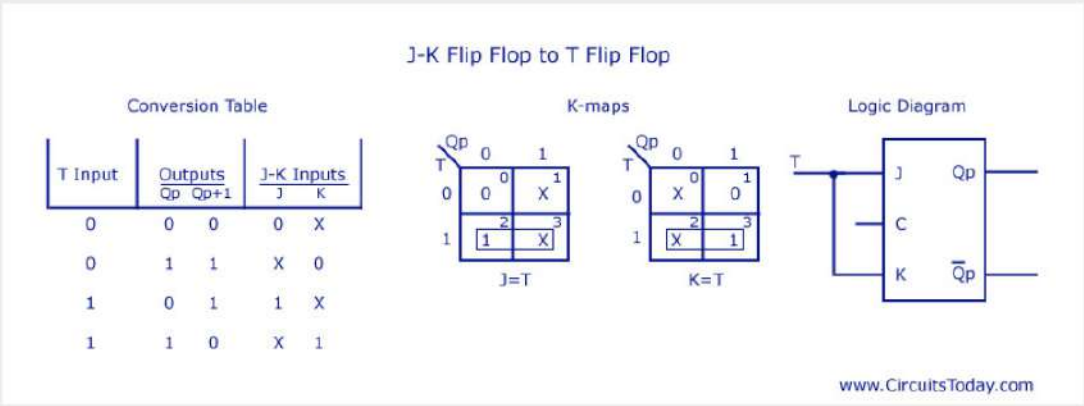
invalid, the values of Qp+1 and D are considered as "don't cares". The logic diagram showing the conversion from D to SR, and the K-map for D in terms of S, R and Qp are shown below.



D Flip Flop to SR Flip Flop

⊡ JK Flip Flop to T Flip Flop

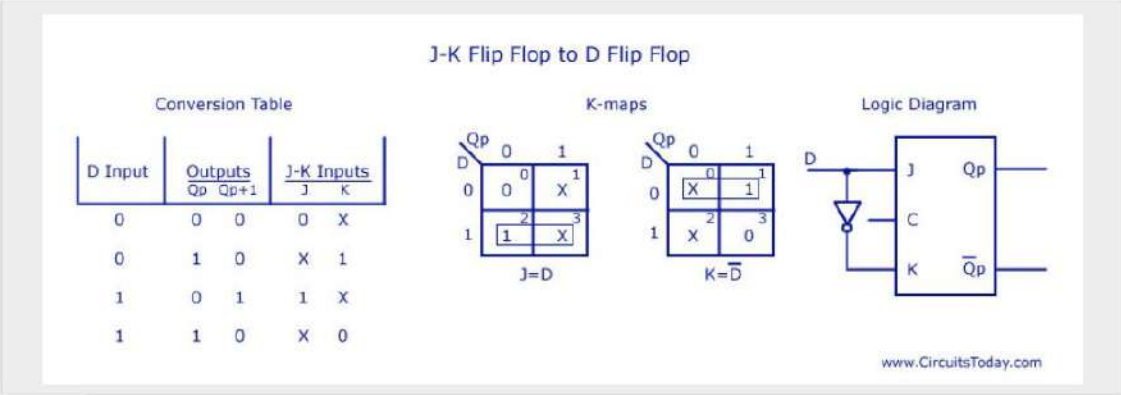
J and K are the actual inputs of the flip flop and T is taken as the external input for conversion. Four combinations are produced with T and Qp. J and K are expressed in terms of T and Qp. The conversion table, K-maps, and the logic diagram are given below.



JK Flip Flop to T Flip Flop

⊡ JK Flip Flop to D Flip Flop

D is the external input and J and K are the actual inputs of the flip flop. D and Qp make four combinations. J and K are expressed in terms of D and Qp. The four combination conversion table, the K-maps for J and K in terms of D and Qp, and the logic diagram showing the conversion from JK to D are given below.

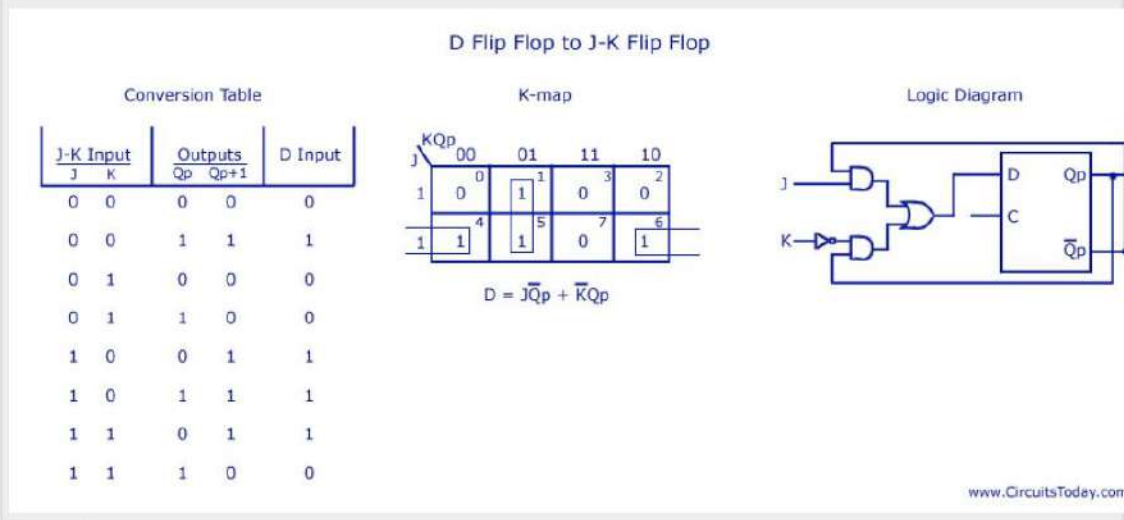


JK Flip Flop to D Flip Flop

⊡ D Flip Flop to JK Flip Flop

In this conversion, D is the actual input to the flip flop and J and K are the external inputs. J, K and Qp make eight possible combinations, as shown in the conversion table below. D is expressed in terms of J, K and Qp.

The conversion table, the K-map for D in terms of J, K and Qp and the logic diagram showing the conversion from D to JK are given in the figure below.



D Flip Flop to JK Flip Flop

Synchronous Counters

Counters are classified into two broad categories according to the way they are clocked; asynchronous and synchronous.

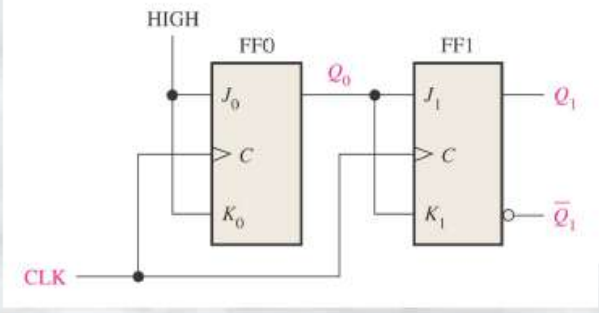
In asynchronous counters, commonly called ripple counters, the first flip-flop is clocked by the external clock pulse and then each successive flip-flop is clocked by the output of the precedingflip-flop.

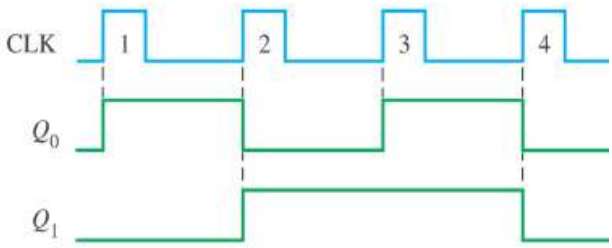
In synchronous counters, the clock input is connected to all of the flip-flops so that they are clocked simultaneously.

With in each of these two categories, counters are classified primarily by the type of sequence, the number of states, or the number of flip-flops in the counter.

Synchronous Counter Operation

- Synchronous counters have a common clock pulse applied simultaneously to all flip-flops.
- A 2-Bit Synchronous Binary Counter





Inputs			Outputs		Comments
J	K	CLK	Q	$\overline{Q}$	
0	0	↑	Q_0	$\overline{Q_0}$	No change
0	1	↑	0	1	RESET
1	0	↑	1	0	SET
1	1	↑	Q_0	Q_0	Toggle

Note that both the J and K inputs are connected together. The flip-flop will toggle when both are a 1 (FF0)

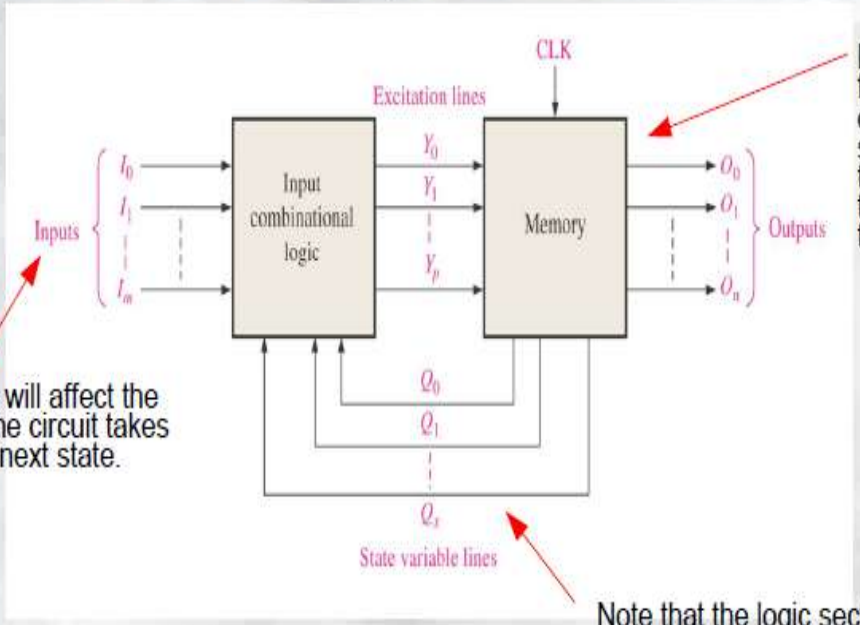
Also the transition at clock pulse 2 works because of propagation delay effects. Q0 is still high on the input of Q1 at the instant clock 2 hits so FF1 changes state. A short time later clock 2 has propagated through FF0 and it goes low.

Design of Synchronous Counters

- We can use synchronous counting circuits to implement state machines.
- State machines are useful in many control and digital applications as they provide the means for taking specific action based upon what state the machine is in and, perhaps, some external event.
- Two types of state machines
  - Moore Circuits – the outputs depend only on the present internal state
  - Mealy Circuits – the output depends on the present state and one or more inputs.

State machines are sequential in that they follow prescribed paths. But the path may vary depending on events.

- General Model of a Sequential Circuit



Memory circuits are flip-flops. They are always in one state or another. The state they are in is called the *present state*. When they change, they go to the *next state*.

Inputs will affect the path the circuit takes to the next state.

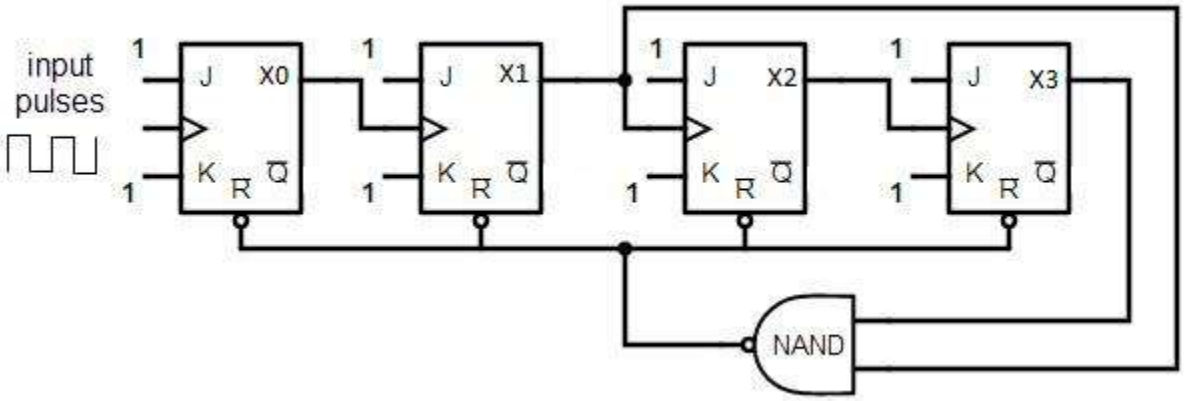
The present state of the output variables Q0 through Qn define the value of the state they are in.

Note that the logic section also looks at the output as well as the input.

Decade Counters

A binary coded decimal (BCD) is a serial digital counter that counts ten digits .And it resets for every new clock input. As it can go through 10 unique combinations of output, it is also called as "Decade counter". A BCD counter can count 0000, 0001, 0010, 1000, 1001, 1010, 1011, 1110, 1111, 0000, and 0001 and so on.

A 4 bit binary counter will act as decade counter by skipping any six outputs out of the 16 (24) outputs. There are some available ICs for decade counters which we can readily use in our circuit, like 74LS90. It is an asynchronous decade counter.

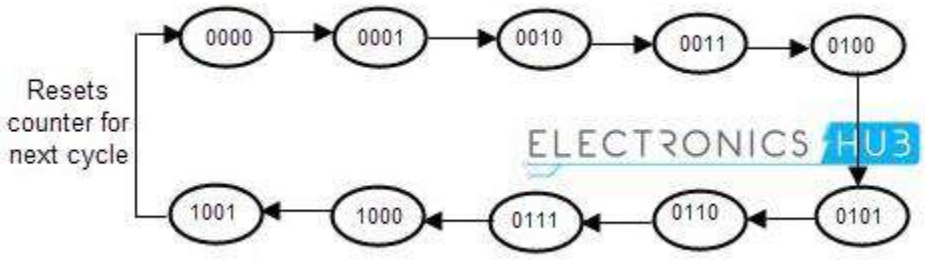


The above figure shows a decade counter constructed with JK flip flop. The J output and K outputs are connected to logic 1. The clock input of every flip flop is connected to the output of next flip flop, except the last one.

The output of the NAND gate is connected in parallel to the clear input 'CLR' to all the flip flops. This ripple counter can count up to 16 i.e. 24.

State Diagram of Decade Counter

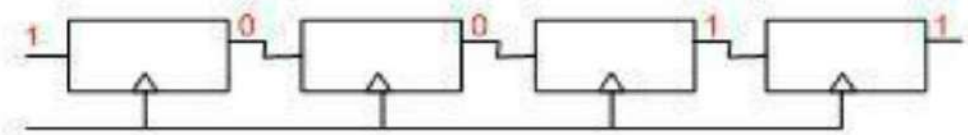
The state diagram of Decade counter is given below. State diagram of Decade counter

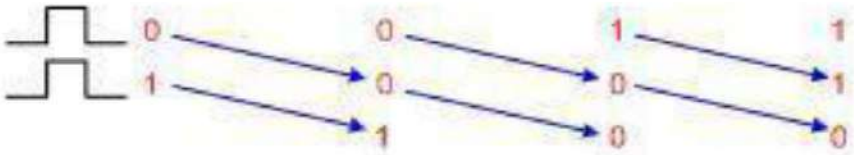


Shift Registers

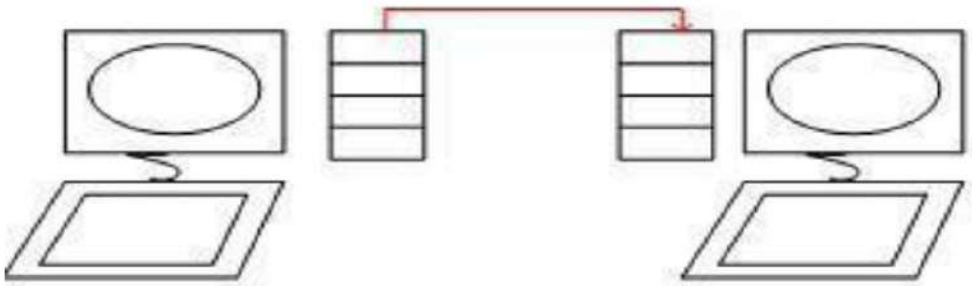
Shift registers are a type of sequential logic circuit, mainly for storage of digital data. They are a group of flip-flops connected in a chain so that the output from one flip-flop becomes the input of the next flip-flop. Most of the registers possess no characteristic internal sequence of states. All the flip-flops are driven by a common clock, and all are set or reset simultaneously. In this chapter, the basic types of shift registers are studied, such as Serial In - Serial Out, Serial In - Parallel Out, Parallel In - Serial Out, Parallel In - Parallel Out, and bidirectional shift registers. A special form of counter - the shift register counter, is also introduced.

Let's observe the values of the flip flops in this shift register for the next couple of clock pulse:



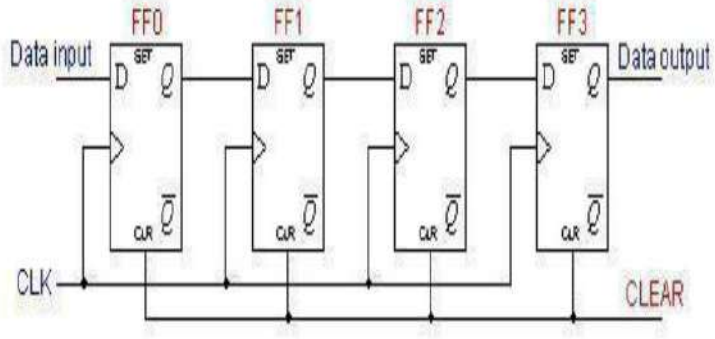


We are actually shifting our data to the right on every clock pulse.Shift registers are widely used in parallel to serial converters which find applications in computer communications.



Serial In - Serial Out Shift Registers

A basic four-bit shift register can be constructed using four D flip-flops, as shown below. The operation of the circuit is as follows. The register is first cleared, forcing all four outputs to zero. The input data is then applied sequentially to the D input of the first flip-flop on the left (FF0). During each clock pulse, one bit is transmitted from left to right. Assume a data word to be 1001. The least significant bit of the data has to be shifted through the register from FF0 to FF3.

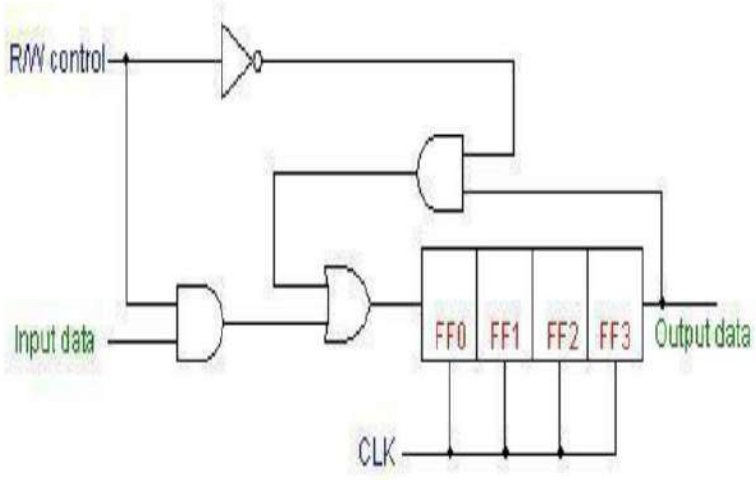


	FF0	FF1	FF2	FF3
CLEAR	0	0	0	0

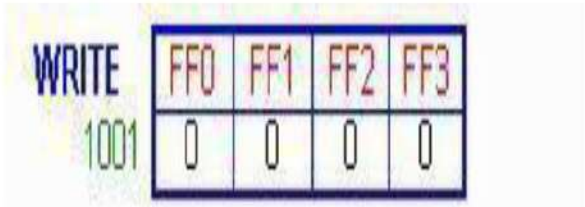
In order to get the data out of the register, they must be shifted out serially. This can be done destructively or non-destructively. For destructive readout, the original data is lost and at the end of the read cycle, all flip-flops are reset to zero.

	FF0	FF1	FF2	FF3
CLEAR	0	0	0	0

To avoid the loss of data, an arrangement for a non-destructive reading can be done by adding two AND gates, an OR gate and an inverter to the system. The construction of this circuit is shown below

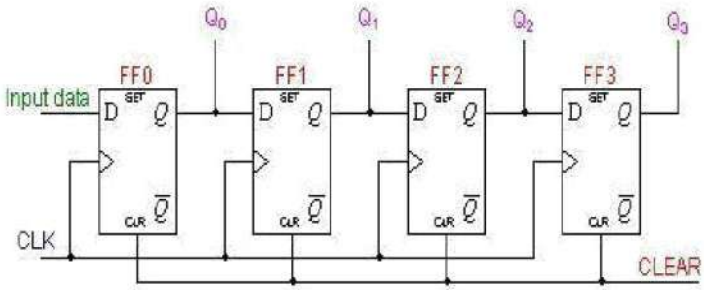


The data is loaded to the register when the control line is HIGH (ie WRITE). The data can be shifted out of the register when the control line is LOW (ie READ). This is shown in the animation below

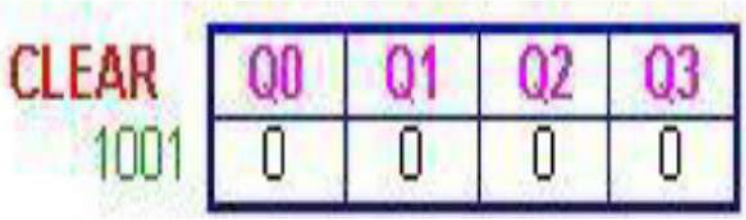


Serial In - Parallel Out Shift Registers

For this kind of register, data bits are entered serially in the same manner as discussed in the last section. The difference is the way in which the data bits are taken out of the register. Once the data are stored, each bit appears on its respective output line, and all bits are available simultaneously. A construction of a four-bit serial in - parallel out register is shown below.

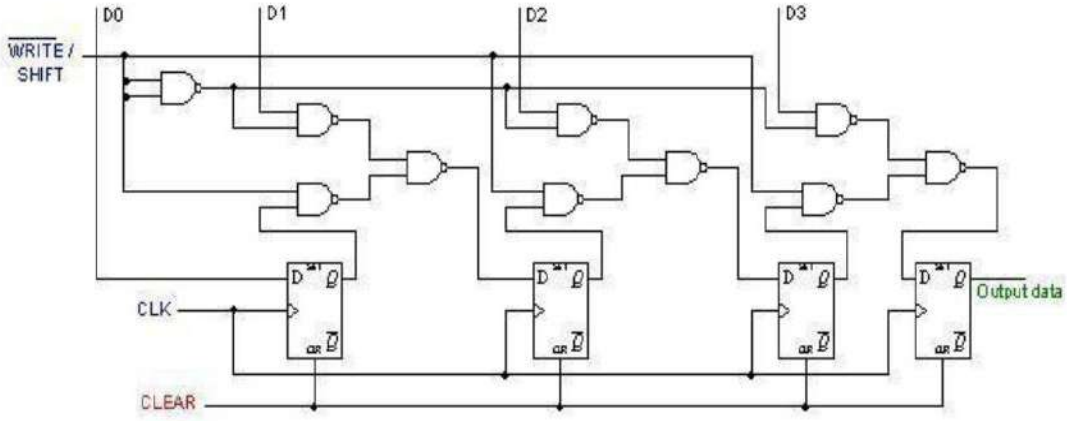


In the animation below, we can see how the four-bit binary number 1001 is shifted to the Q outputs of the register.



Parallel In - Serial Out Shift Registers

A four-bit parallel in - serial out shift register is shown below. The circuit uses D flip-flops and NAND gates for entering data (ie writing) to the register.

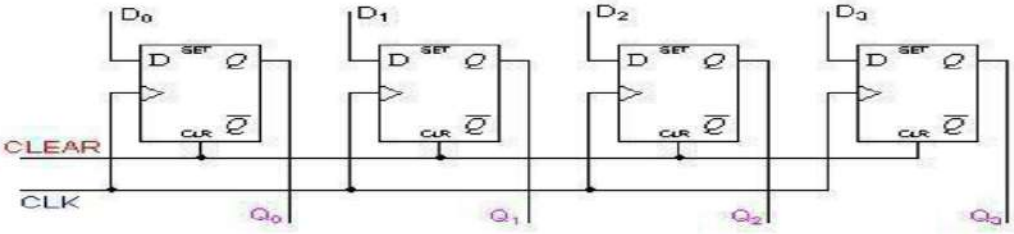


D0, D1, D2 and D3 are the parallel inputs, where D0 is the most significant bit and D3 is the least significant bit. To write data in, the mode control line is taken to LOW and the data is clocked in. The data can be shifted when the mode control line is HIGH as SHIFT is active high. The register performs right shift operation on the application of a clock pulse, as shown in the animation below.

	Q0	Q1	Q2	Q3
CLEAR	0	0	0	0

Parallel In - Parallel Out Shift Register

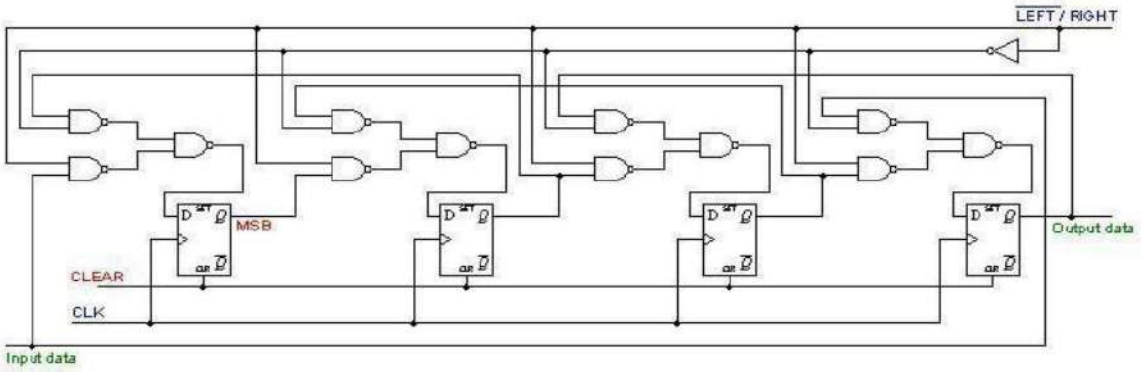
For parallel in - parallel out shift registers, all data bits appear on the parallel outputs immediately following the simultaneous entry of the data bits. The following circuit is a four-bit parallel in - parallel out shift register constructed by D flip-flops.



The D's are the parallel inputs and the Q's are the parallel outputs. Once the register is clocked, all the data at the D inputs appear at the corresponding Q outputs simultaneously.

Bidirectional Shift Registers

The registers discussed so far involved only right shift operations. Each right shift operation has the effect of successively dividing the binary number by two. If the operation is reversed (left shift), this has the effect of multiplying the number by two. With suitable gating arrangement a serial shift register can perform both operations. A *bidirectional*, or *reversible*, shift register is one in which the data can be shift either left or right. A four-bit bidirectional shift register using D flip-flops is shown below.


LEFT / RIGHT
MSB
CLEAR
CLK
Input data
Output data

MEMORIES - ROM Architecture, Types of ROMS & Applications

A ROM is essentially a memory device in which permanent binary information is stored. The binary information must be specified by the designer and is then embedded in the unit to form the required interconnection pattern. Once the pattern is established it stays within the unit even when power is turned off and on again.

A block diagram of ROM is shown in the Figure 1. It consists of k inputs and n outputs. The inputs provide the address for the memory and the outputs give the data bits of the stored word which is selected by the address. The number of words in a ROM is determined from the fact that k address input lines are needed to specify 2^k words.

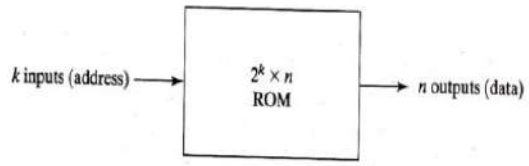


Fig 1 ROM Block Diagram

ROM does not have data inputs because it does not have a write operation.

Consider for example a 32 x 8 ROM. The unit consists of 32 words of 8 bits each. There are five input lines that form the binary numbers from 0 through 31 for the address. The Figure 2 shows the internal logic construction of the ROM. The five inputs are decoded into 32 distinct outputs by means of a 5 x 32 decoder. Each output of the decoder represents a memory address. The 32 outputs of the decoder are connected to each of the eight OR gates.

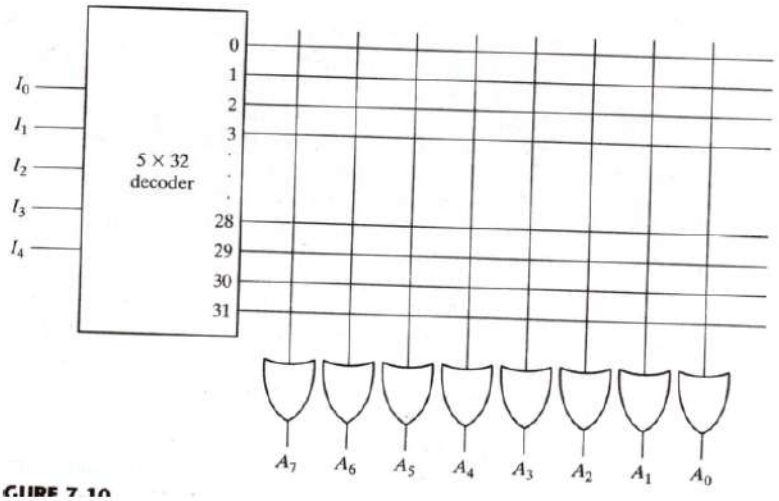


Fig 2 Internal Logic of a 32 X 8 ROM

The diagram shows the array logic convention used in complex circuits . Each OR gate must be considered as having 32 inputs. Each output of the decoder is connected to one of the inputs of each OR gate. Since each OR gate has 32 input connections and there are 8 OR gates, the ROM contains 32 x 8 = 256 internal connections.

In general, a 2^k x n ROM will have an internal k x 2^k decoder and n OR gates. Each OR gate has 2^k inputs, which are connected to each of the outputs of the decoder.

The 256 intersections of the above figure are programmable. A programmable connection

Inputs					Outputs				
I4	I3	I2	I1	I0	A7	A6	A5	A4	A3
0	0	0	0	0	1	0	1	1	0
0	0	0	0	1	0	0	0	1	1
0	0	0	1	0	1	1	0	0	0
0	0	0	1	1	1	0	1	1	0
		⋮					⋮		
1	1	1	0	0	0	0	0	0	1
1	1	1	0	1	1	1	1	0	0
1	1	1	1	0	0	1	0	0	1
1	1	1	1	1	0	0	1	1	0

Table 1 ROM Truth Table

Between the lines is logically equivalent to a switch that can be altered to either be close or open .The programmable intersection between two lines is sometimes called cross point.

The internal binary storage of a ROM is specified by a truth table that shows the word content in each address. The Table 1 shows the five inputs under which are listed all 32 addresses. At each address, there is stored a word of 8 bits, which is listed under the outputs columns .The table shows only the first four and the last four words in the ROM. The complete table must include the list of all 32 words.

The hardware procedure that programs the ROM results in blowing fuse links according to a given truth table. The programming of ROM according to the truth table is given in Table 1 , results in the configuration shown in Figure 2.

Every 0 listed in the truth table specifies a no connection and every 1 listed specifies a path that is obtained by a connection. The four 0's in the word are programmed by blowing the fuse links between output 3 of the decoder and the inputs of the OR gates associated with outputs A_6 , A_3, A_2 and A_0. The four 1's in the word are marked in the diagram with a X to denote a connection in place of a dot used for permanent connection in logic diagrams.

When the input of the ROM is 00011, all the outputs of the decoder are 0 except for output 3, which is at logic 1. The signal equivalent to logic 1 at decoder output 3 propagates through the connections to the OR gate outputs of A_7 , A_5, A_4 and A_1.The other four outputs remain at 0. The result is that the stored word 10110010 is applied to the eight data outputs.

COMBINATIONAL PLDs

The PROM is a combinational programmable logic device (PLD). A combinational PLD is an integrated circuit with programmable gates divided into an AND array and an OR array to provide an AND-OR sum of product implementation.

There are three major types of combinational PLDs and they differ in the placement of the programmable connections in the AND-OR array. The Figure 3 shows the configuration of three PLDs.

The PROM has a fixed AND array constructed as a decoder and programmable OR array. The programmable OR gates implement the Boolean functions in sum of minterms.

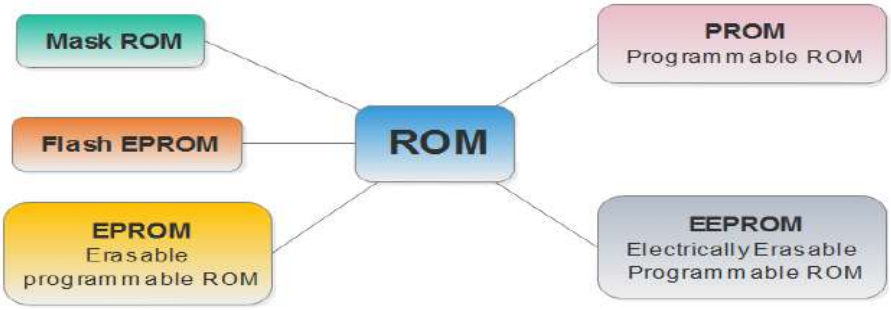
Types of ROM:

Although all ROM basically serves the same purpose, there are a few different types commonly in use today.

Understanding the different types of ROMs will also help you understand how they're used for different applications, and which type may apply to your application.

Some of them are:

1. PROM (Programmable ROM)
2. EPROM (Erasable Programmable ROM)
3. EEPROM (electrically erasable programmable ROM)
4. Flash EPROM
5. Mask ROM



1. PROM (programmable ROM) and OTP

PROM refers to the kind of ROM that the user can burn information into. In other words, PROM is a user-programmable memory.

For every bit of the PROM, there exists a fuse. PROM is programmed by blowing the fuses. If the information burned into PROM is wrong, that PROM must be discarded since its internal fuses are blown permanently. For this reason, PROM is also referred to as OTP (One Time Programmable).

Programming ROM, also called burning ROM, requires special equipment called a ROM burner or ROM programmer.

2. EPROM (erasable programmable ROM) and UV-EPROM

EPROM was invented to allow making changes in the contents of PROM after it is burned.

In EPROM, one can program the memory chip and erase it thousands of times. This is especially necessary during the development of the prototype of a microprocessor-based project.

A widely used EPROM is called UV-EPROM, where UV stands for ultraviolet. The only problem with UV-EPROM is that erasing its contents can take up to 20 minutes.

All UV-EPROM chips have a window through which the programmer can shine ultraviolet (UV) radiation to erase the chip's contents. For this reason, EPROM is also referred to as UV-erasable EPROM or simply UV-EPROM.

Programming a UV-EPROM

To program a UV-EPROM chip, the following steps must be taken:

1. Its contents must be erased. To erase a chip, remove it from its socket on the system board and place it in EPROM erasure equipment to expose it to UV radiation for 5—20 minutes.
2. Program the chip. To program a UV-EPROM chip, place it in the ROM burner (programmer). To burn code or data into EPROM, the ROM burner uses 12.5 volts or higher, depending on the EPROM type. This voltage is referred to as V_{pp} in the UV-EPROM data sheet.
3. Place the chip back into its socket on the system board.

As can be seen from the above steps, not only is there an EPROM programmer (burner), but there is also separate EPROM erasure equipment.

The main problem, and indeed the major disadvantage of UV-EPROM, is that it cannot be erased and programmed while it is in the system board. To provide a solution to this problem, EEPROM was invented. '

3. EEPROM (electrically erasable programmable ROM)

EEPROM has several advantages over EPROM, such as the fact that its method of erasure is electrical and therefore instant. as opposed to the 20-minute erasure time required for UV-EPROM.

In addition, in EEPROM one can select which byte to be erased, in contrast to UV-EPROM, in which the entire contents of ROM are erased.

However, the main advantage of EEPROM is that one can program and erase its contents while it is still in the system board. it does not require physical removal of the

memory chip from its socket. In other words, unlike UV-EPROM, EEPROM does not require an external erasure and programming device.

To utilize EEPROM fully, the designer must incorporate the circuitry to program the EEPROM into the system board. In general, the cost per bit for EEPROM is much higher than for UV-EPROM.

4. Flash memory EPROM

Since the early 1990s, Flash EPROM has become a popular user-programmable memory chip. and for good reasons.

1. First, the erasure of the entire contents takes less than a second, or one might say in a flash, hence its name, Flash memory.
2. In addition, the erasure method is electrical, and for this reason, it is sometimes referred to as Flash EEPROM. To avoid confusion, it is commonly called Flash memory.

The major difference between EEPROM and Flash memory is that when Flash memory's contents are erased, the entire device is erased, in contrast to EEPROM, where one can erase a desired byte.

Although in many Flash memories recently made available the contents are divided into blocks and the erasure can be done block by block, unlike EEPROM, Flash memory has no byte erasure option.

Because Flash memory can be programmed while it is in its socket on the system board, it is widely used to upgrade the BIOS ROM of the PC. Some designers believe that Flash memory will replace the hard disk as a mass storage medium.

This would increase the performance of the computer tremendously since Flash memory is semiconductor memory with access time in the range of 100 ns compared with disk access time in the range of tens of milliseconds. For this to happen, Flash memory's program/erase cycles must become infinite, just like hard disks.

Program/erase cycle refers to the number of times that a chip can be erased and reprogrammed before it becomes unusable. At this time, the program/erase cycle is 100,000 for Flash and EEPROM, 1000 for UV-EPROM, and infinite for RAM and disks.

5. Mask ROM

Mask ROM refers to a kind of ROM in which the contents are programmed by the IC manufacturer. In other words, it is not a user-programmable ROM.

The term mask is used in IC fabrication. Since the process is costly, mask ROM is used when the needed volume is high (hundreds of thousands) and it is absolutely certain that the contents will not change.

It is common practice to use UV-EPROM or Flash for the development phase of a project, and only afier the code/data have been finalized is the mask version of the product ordered.

The main advantage of mask ROM is its cost, since it is significantly cheaper than other kinds of ROM, but if an error is found in the data/code, the entire batch must be thrown away. It must be noted that all ROM memories have 8 bits for data pins; therefore, the organization is x8.

Applications of Mask Read Only Memory (MROM)

The Mask Read-Only Memory (MROM) are used for:

- Network Operating Systems.
- Server Operating Systems.
- Storing fonts for laser printers.
- Storing sound data in electronic musical instruments.

Applications of Programmable Read Only Memory (PROM)

The Programmable ROM (PROM) are used in:

- Mobile Phones for providing User Specific Selections.
- Video game consoles
- Implantable Medical devices.
- Radio-Frequency Identification (RFID)tags.
- High definition Multimedia Interfaces(HDMI)

Applications of Erasable Programmable Read Only Memory (EPROM)

The applications of Erasable Programmable ROM (EPROM) includes:

- As program storage chip in Micro controllers.
- For debugging.
- For program development.
- As BIOS chip in computers.
- As program storage chip in modem, video card and many electronic gadgets.

Applications of Electrically Erasable Programmable Read Only Memory (EEPROM)

The applications of Electrically Erasable Programmable ROM (EPROM) includes:

- As BIOS chip in computers
- As storage for re-programmable calibration information in test-equipment.

- As storage for in-built self learning functionality in remote operated transmitters.

Applications of Flash Read Only Memory (Flash ROM)

The applications of Flash Read-Only Memory (Flash ROM) are:

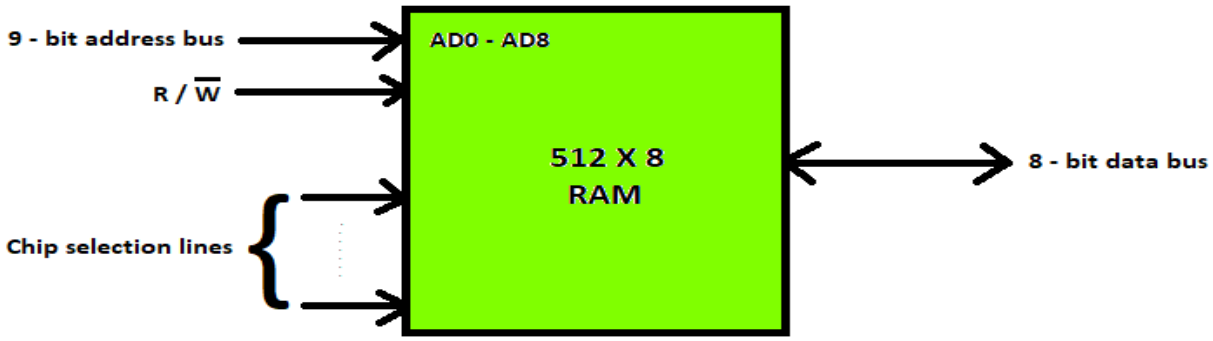
- The latest technology computers use BIOS stored on a flash memory chip, called as flash BIOS.
- Modems, pen drives, small cards use flash ROM.

RAM Architecture:

RAM(Random Access Memory) is a part of computer's Main Memory which is directly accessible by CPU. RAM is used to Read and Write data into it which is accessed by CPU randomly. RAM is volatile in nature, it means if the power goes off, the stored information is lost. RAM is used to store the data that is currently processed by the CPU. Most of the programs and data that are modifiable are stored in RAM.

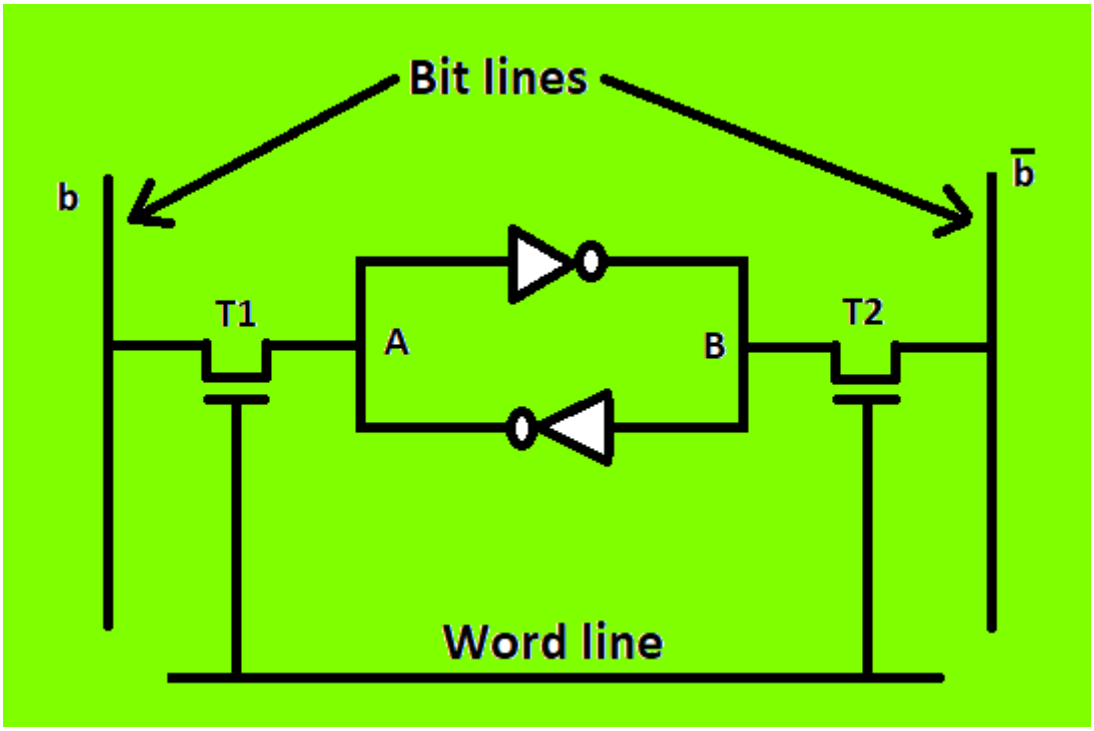
Integrated RAM chips are available in two form:

1. SRAM(Static RAM)
2. DRAM(Dynamic RAM)



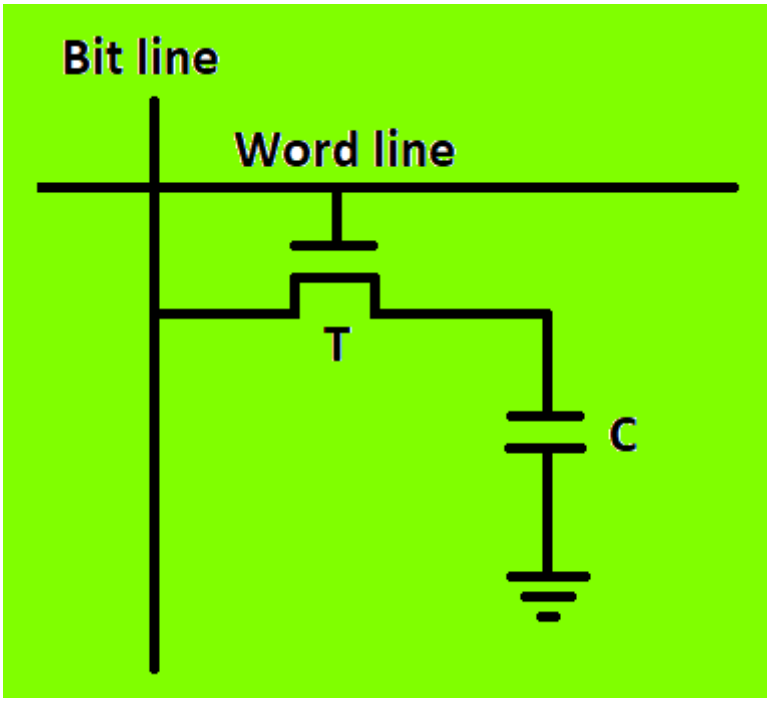
SRAM Memory Cell: Static memories(SRAM) are memories that consist of circuits capable of retaining their state as long as power is on. Thus this type of memories is called volatile memories. The below figure shows a cell diagram of SRAM. A latch is formed by two inverters connected as shown in the figure. Two transistors T1 and T2 are used for connecting the latch with two bit lines. The purpose of these transistors is to act as switches that can be opened or closed under the control of the word line, which is controlled by the address decoder. When the word line is at 0-level, the transistors are turned off and the latch remains its information. For

example, the cell is at state 1 if the logic value at point A is 1 and at point B is 0. This state is retained as long as the word line is not activated.



For **Read operation**, the word line is activated by the address input to the address decoder. The activated word line closes both the transistors (switches) T1 and T2. Then the bit values at points A and B can transmit to their respective bit lines. The sense/write circuit at the end of the bit lines sends the output to the processor.
For **Write operation**, the address provided to the decoder activates the word line to close both the switches. Then the bit value that to be written into the cell is provided through the sense/write circuit and the signals in bit lines are then stored in the cell.

DRAM Memory Cell: Though SRAM is very fast, but it is expensive because of its every cell requires several transistors. Relatively less expensive RAM is DRAM, due to the use of one transistor and one capacitor in each cell, as shown in the below figure., where C is the capacitor and T is the transistor. Information is stored in a DRAM cell in the form of a charge on a capacitor and this charge needs to be periodically recharged.
For storing information in this cell, transistor T is turned on and an appropriate voltage is applied to the bit line. This causes a known amount of charge to be stored in the capacitor. After the transistor is turned off, due to the property of the capacitor, it starts to discharge. Hence, the information stored in the cell can be read correctly only if it is read before the charge on the capacitors drops below some threshold value.



Difference between SRAM and DRAM

Below table lists some of the differences between SRAM and DRAM:

SRAM	DRAM
1. SRAM has lower access time, so it is faster compared to DRAM.	1. DRAM has higher access time, so it is slower than SRAM.
2. SRAM is costlier than DRAM.	2. DRAM costs less compared to SRAM.
3. SRAM requires constant power supply, which means this type of memory consumes more power.	3. DRAM offers reduced power consumption, due to the fact that the information is stored in the capacitor.
4. Due to complex internal circuitry, less storage capacity is available compared to the same physical size of DRAM memory chip.	4. Due to the small internal circuitry in the one-bit memory cell of DRAM, the large storage capacity is available.
5. SRAM has low packaging density.	5. DRAM has high packaging density.